

Bases de Datos

Clase 13: ORM – NoSQL y full-text search & retrieval

Lo que hemos visto

Contenidos del curso

	Contenido	Proyecto
✓	El modelo relacional	
✓	Consultas en álgebra relacional	
✓	Modelamiento de un problema con el modelo E/R	
✓	Transformación de un MER a un esquema relacional	
✓	Identificación de dependencias funcionales y anomalías	
✓	Formas normales 1FN, 2FN, 3FN y FNBC	E1
✓	SQL	E2
✓	Vistas, triggers y stored procedures	E3
✓	Transacciones, ACID, schedules y locks	E3
✓	Recuperación ante fallas	
✓	Índices	
✓	Algoritmos: selección y join	

Hasta aquí: bases de datos relacionales. **Hoy: NoSQL**

Datos según su grado de estructuración

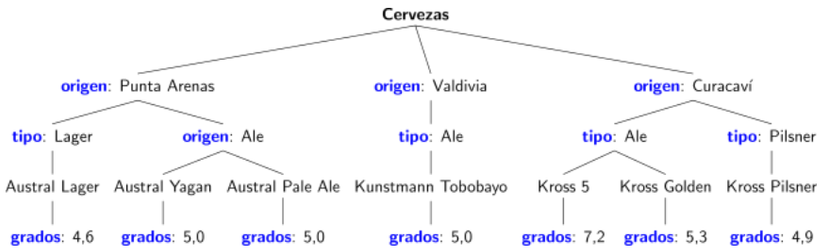
Clasificación de los datos

Relacional
(SQL, CSV, …)

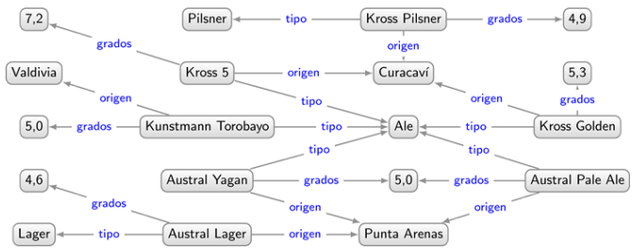
Cervezas

nombre	tipo	grados	ciudad-origen
Austral Lager	Lager	4,6	Punta Arenas
Austral Yagan	Ale	5,0	Punta Arenas
Austral Pale Ale	Ale	5,0	Punta Arenas
Kuntsmann Torobayo	Ale	5,0	Valdivia
Kross 5	Ale	7,2	Curacaví
Kross Golden	Ale	5,3	Curacaví
Kross Pilsner	Pilsner	4,9	Curacaví

Árboles
(XML, JSON, …)



Grafos
(RDF, Prop. Gs, …)



Texto enriquecido
(HTML, Word, …)

```
<ul>
  <li>Austral Lager [Lager] 4,6% de Punta Arenas</li>
  <li>Austral Yagan [Ale] 5% de Punta Arenas</li>
  <li>Austral Pale Ale [Ale] 5% de Punta Arenas</li>
  <li>Kunstsmann Torobayo [Ale] 5% de Valdivia</li>
  <li>Kross 5 [Ale] 7,2% de Curacaví</li>
  <li>Kross Golden [Ale] 5,3% de Curacaví</li>
  <li>Kross Pilsner [Pilsner] 4,9% de Curacaví</li>
</ul>
```

Texto plano

Hay mucha variedad en las cervezas locales de Chile. La sede de la marca "Austral" se encuentra en Punta Arenas. Austral fabrica una amplia gama de cervezas, incluyendo Larger (4,6%), Yagan (un ale de 5%) y un Pale Ale (5%). La cerveza de marca "Kunstmann" también es popular, en particular su cerveza "Torobayo" (un ale de 5% elaborado en Valdiva). La marca Kross, basada en Curacaví, también tiene una gama de cervezas populares como, por ejemplo, Kross 5 (un ale fuerte de 7,2%) Kross Golden (un ale de 5,3%) y Kross Pilsner (4,9%).

Estructurados

Semi-estructurados

No estructurados

Bases de datos relacionales y NoSQL

Más allá del modelo relacional

Relacionales

- Mucha estructura (un esquema fijo)
- Muchas garantías (ACID)
- Generalmente centralizadas (residen en un servidor)

NoSQL

También denominadas *Not Only SQL*. Término común para designar las bases de datos surgidas desde 1998:

- Menos restricciones que el modelo relacional
- Menos esquema
- Menos garantías de consistencia
- Más adecuadas para la **distribución**

Sin embargo, existen más tipos de bases de datos: **las NoSQL**

SQL vs. NoSQL

Comparación (1/2)

Aspecto	SQL (relacional)	NoSQL (no relacional)
Modelo de datos	Estructurado, basado en tablas con esquemas y relaciones fijas	Modelos de datos flexibles (documentos, clave-valor, columna, grafo)
Esquema	Rígido; debe definirse previamente con una estructura estricta	Sin esquema o flexible; permite datos dinámicos y no estructurados
Escalabilidad	Escalado vertical (scale-up)	Escalado horizontal (scale-out) entre nodos distribuidos
Consistencia	Consistencia fuerte con transacciones ACID	Consistencia eventual, ajustable en algunos sistemas (por ejemplo, MongoDB)
Lenguaje de consulta	SQL (estandarizado y potente para consultas complejas)	Varía (por ejemplo, lenguaje de consulta de MongoDB, Cassandra Query Language)
Soporte de transacciones	ACID completo (atomicidad, consistencia, aislamiento, durabilidad)	Modelo BASE (básicamente disponible, estado suave, consistencia eventual); algunos soportan ACID en contextos limitados

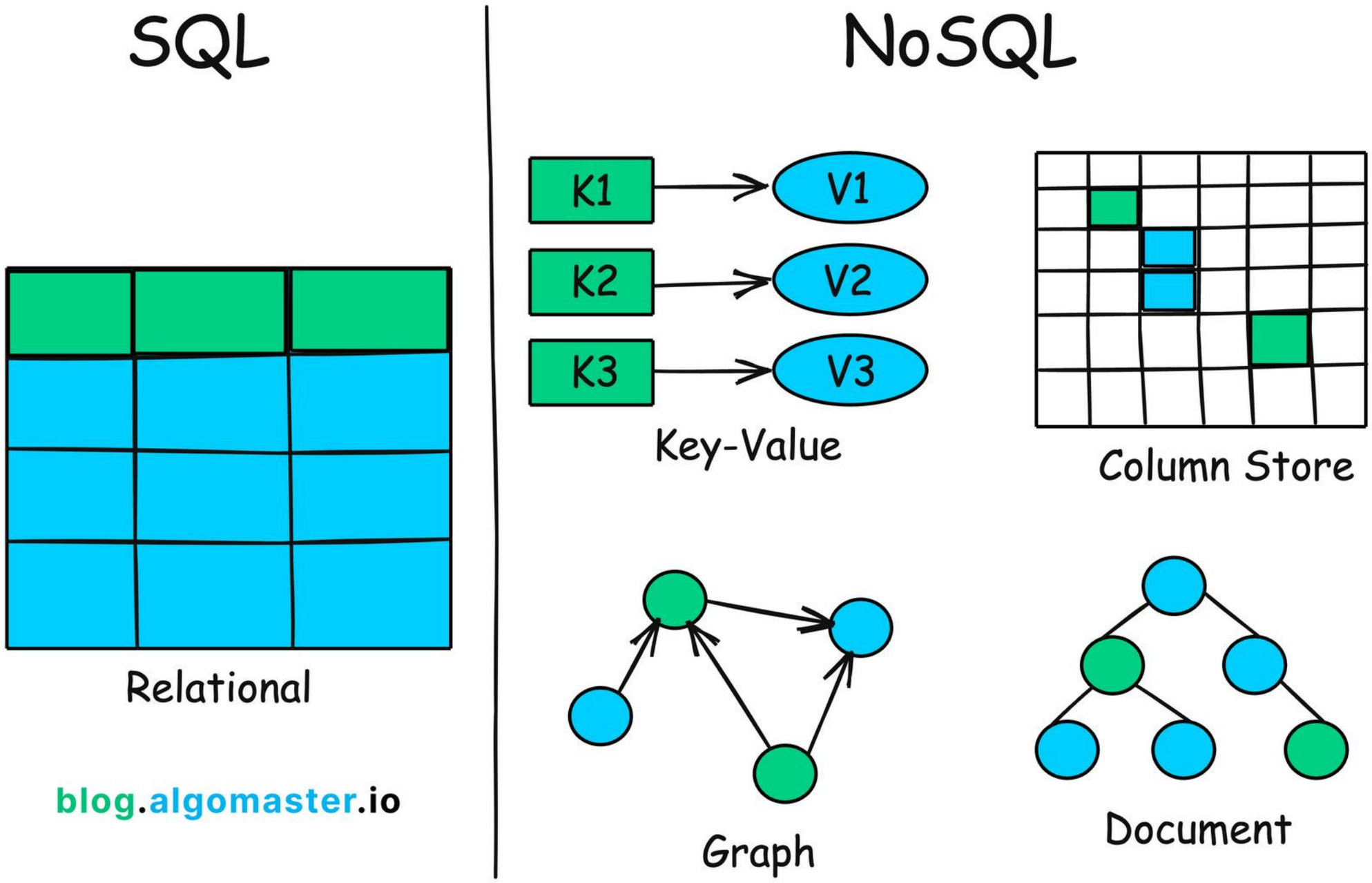
SQL vs. NoSQL

Comparación (2/2)

Aspecto	SQL (relacional)	NoSQL (no relacional)
Rendimiento	Optimizado para consultas complejas y relaciones; más lento en escrituras a gran escala	Optimizado para lecturas y escrituras de alto rendimiento, especialmente en sistemas distribuidos
Casos de uso	Datos estructurados y aplicaciones que requieren alta consistencia (por ejemplo, sistemas financieros, ERP, CRM)	Datos no estructurados o semiestructurados y sistemas distribuidos a gran escala (big data, analítica en tiempo real, redes sociales)
Desafíos de escalado	Difícil de fragmentar o particionar en varios servidores	Se pueden fragmentar y distribuir de forma sencilla
Integridad de datos	Alta; las relaciones y la integridad se aseguran mediante restricciones	Varía; la integridad suele gestionarse a nivel de aplicación
Uniones y relaciones	Permite uniones complejas entre tablas y relaciones de clave foránea	Las relaciones se gestionan de forma embebida (documentos) o de otra forma (bases de datos de grafos)
Ejemplos	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, DynamoDB, Redis, Neo4j

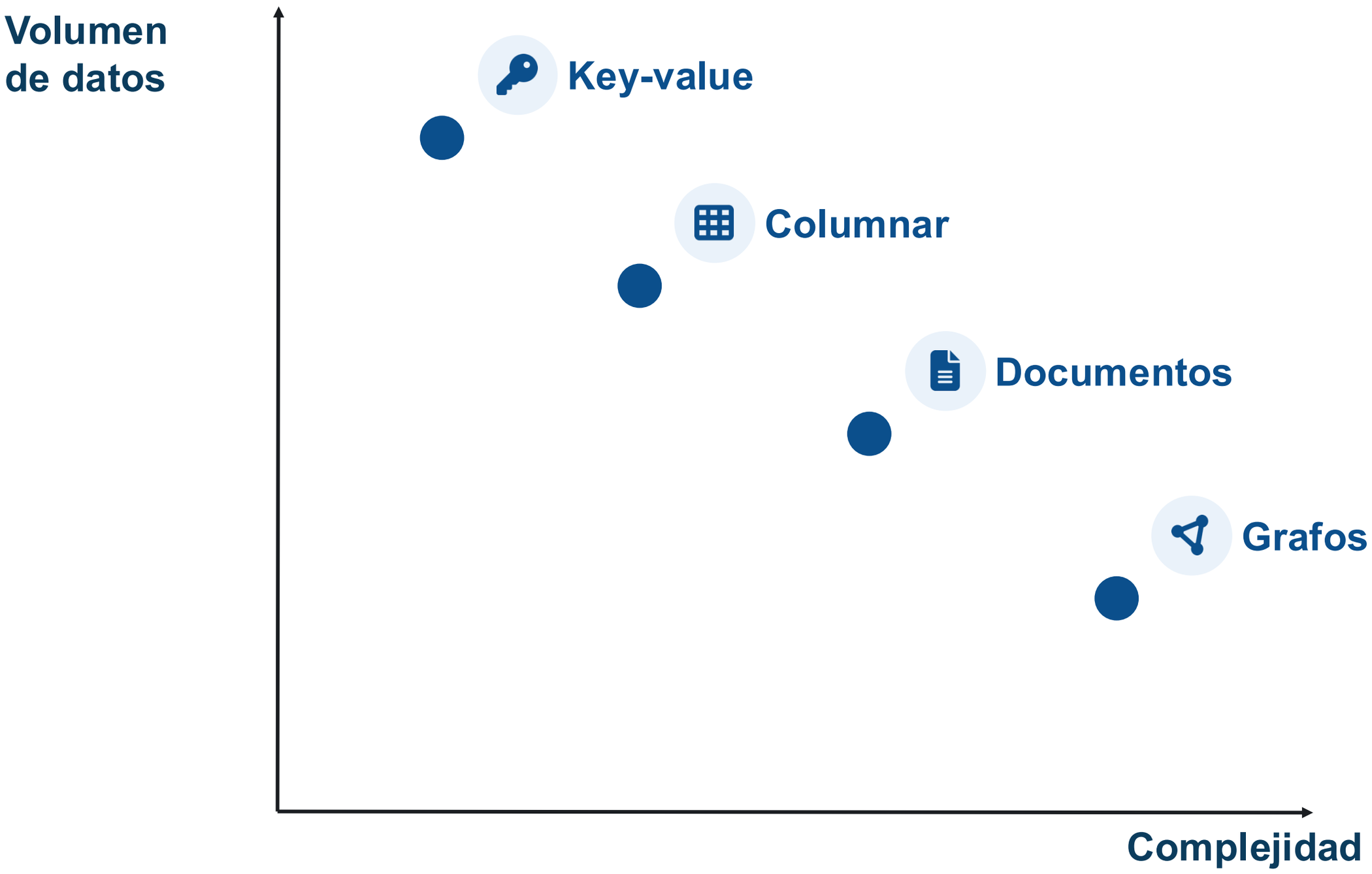
Modelos SQL y NoSQL

Representación gráfica



Sabores de NoSQL

Volumen de datos vs. complejidad



Popularidad de las bases de datos

Ranking DB-Engines

440 systems in ranking, October 2026

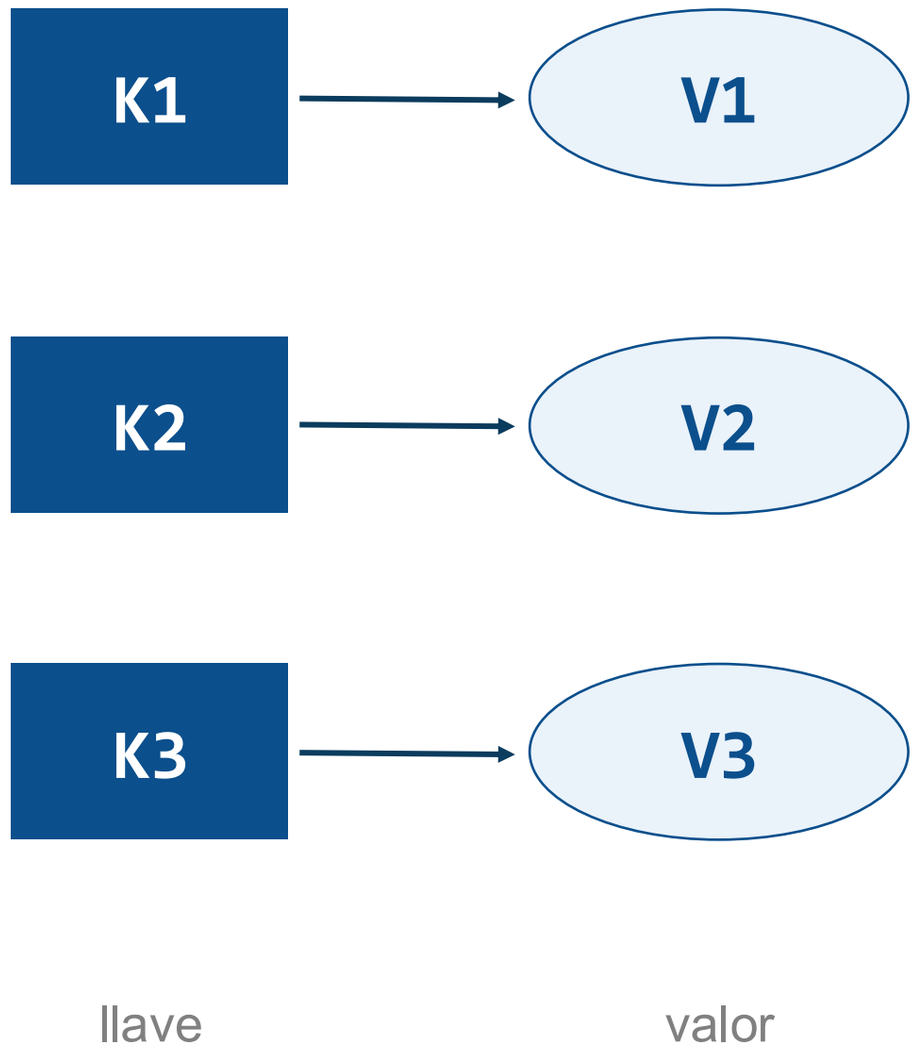
Rank Oct 2026	Rank Sep 2026	Rank Oct 2025	DBMS	Database Model	Score Oct 2026	Score Sep 2026	Score Oct 2025
1.	1.	1.	Oracle	Relational, Multi-model	1119.79	-2.48	-92.98
2.	2.	2.	MySQL	Relational, Multi-model	837.96	-6.85	-41.70
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	694.47	-4.22	-20.58
4.	4.	4.	PostgreSQL	Relational, Multi-model	688.75	+5.50	+45.56
5.	5.	5.	MongoDB	Document, Multi-model	374.64	-6.66	+6.63
6.	6.	6.	Snowflake	Relational	221.30	+9.69	+22.65
7.	7.	↑ 8.	Databricks	Multi-model	171.24	+2.90	+42.44
8.	8.	↓ 7.	Redis	Key-value, Multi-model	156.09	-1.97	+13.76
9.	9.	9.	IBM Db2	Relational, Multi-model	106.29	-4.28	-16.08
10.	↑ 12.	↑ 11.	Apache Cassandra	Wide column, Multi-model	98.86	+4.90	-6.30
11.	↓ 10.	↑ 12.	SQLite	Relational	95.73	-1.32	-8.82
12.	↓ 11.	↓ 10.	Elasticsearch	Multi-model	94.11	-1.39	-22.55
13.	13.	13.	MariaDB	Relational, Multi-model	75.60	+1.18	-12.16
14.	↑ 16.	↑ 17.	Apache Hive	Relational	69.58	-0.53	-5.64
15.	↓ 14.	↑ 18.	Splunk	Search engine	69.25	-2.43	-4.62
16.	↓ 15.	16.	Microsoft Azure SQL Database	Relational, Multi-model	68.55	-2.29	-6.94
17.	17.	↓ 14.	Microsoft Access	Relational	63.32	+0.27	-17.47
18.	18.	↓ 15.	Amazon DynamoDB	Multi-model	53.41	-2.94	-22.49
19.	19.	19.	Google BigQuery	Relational	49.78	-2.80	-13.43
20.	20.	20.	Neo4j	Graph	48.88	+1.72	-3.63
21.	21.	21.	Apache Solr	Search engine, Multi-model	32.41	-1.62	-2.90
22.	22.	↑ 24.	SAP HANA	Relational, Multi-model	32.40	-0.79	+0.75
23.	23.	↓ 22.	Teradata	Relational, Multi-model	31.23	+0.05	-1.88
24.	24.	↓ 23.	FileMaker	Relational	29.80	-0.35	-3.03
25.	↑ 26.	↑ 30.	ClickHouse	Relational, Multi-model	26.18	+0.35	+6.00
26.	↓ 25.	↓ 25.	SAP Adaptive Server	Relational, Multi-model	24.87	-1.19	-2.28
27.	27.	↑ 29.	PostGIS	Spatial, Multi-model	23.96	+0.10	+2.91
28.	↑ 30.	↑ 32.	OpenSearch	Multi-model	22.72	+0.61	+3.42
29.	↓ 28.	↓ 26.	Apache Spark (SQL)	Relational	22.37	-0.95	-1.41
30.	↑ 31.	↓ 27.	Microsoft Azure Cosmos DB	Multi-model	21.38	+1.03	-1.33
31.	↓ 29.	↓ 28.	InfluxDB	Time Series, Multi-model	21.22	-1.08	-0.69
32.	32.		Microsoft Fabric	Relational	18.99	-0.42	
33.	↑ 34.	33.	Firebird	Relational	18.36	+0.93	+0.94
34.	↓ 33.	↓ 31.	Apache HBase	Wide column	18.22	-0.34	-1.95
35.	35.	↑ 37.	Amazon Redshift	Relational	17.08	+0.62	+2.49
36.	36.	↑ 38.	Microsoft Azure Synapse Analytics	Relational	15.90	-0.29	+2.09
37.	37.	↓ 35.	Firebase Realtime Database	Document	15.55	+0.59	+0.28
38.	↑ 39.	↓ 36.	Informix	Relational, Multi-model	13.29	+0.41	-1.63
39.	↓ 38.	↓ 34.	Memcached	Key-value	12.96	-0.10	-2.52
40.	40.	↑ 41.	Apache Flink	Relational	11.57	+0.06	-0.07

Fuente:
https://db-engines.com/en/ranking

BD key-value

Definición

- Independientes del esquema
- La arquitectura almacena la información por medio de pares
- Cada par se compone de una llave (identificador) y un valor



BD key-value

Operaciones cruciales

```
put(key, value)
get(key)
delete(key)
```

Ejemplo: capitales por país

Key	Value
Chile	Santiago
Inglaterra	Londres
Escocia	Edimburgo
Francia	París
Alemania	Berlín
…	…

BD key-value

Características

- Son grandes tablas de hash persistentes
- Categoría difusa: muchas aplicaciones de otros tipos de BD utilizan key-value y hashing en cierta medida
- Ejemplo más importante: Amazon Dynamo; otro ejemplo es Redis

BD key-value

Valores arbitrarios

Un valor puede representar cualquier tipo de dato. Por ejemplo, carros de compra:

cartID	value
11789	usrID: "Juan", item: "Magic the Gathering Deck", value: …
12309	usrID: "Domagoj", item: "APEX XTX50 regulator set", value: …
…	…

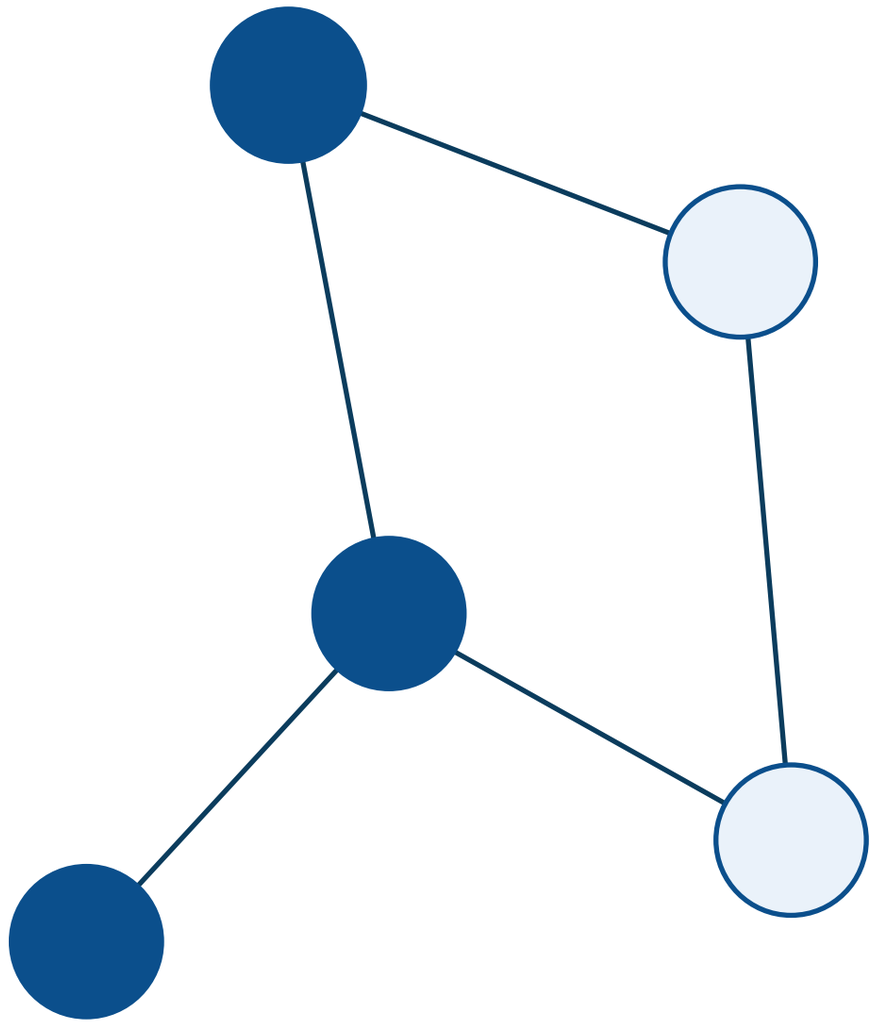


La llave identifica el carro; el valor almacena su contenido completo.

BD de grafos y RDF

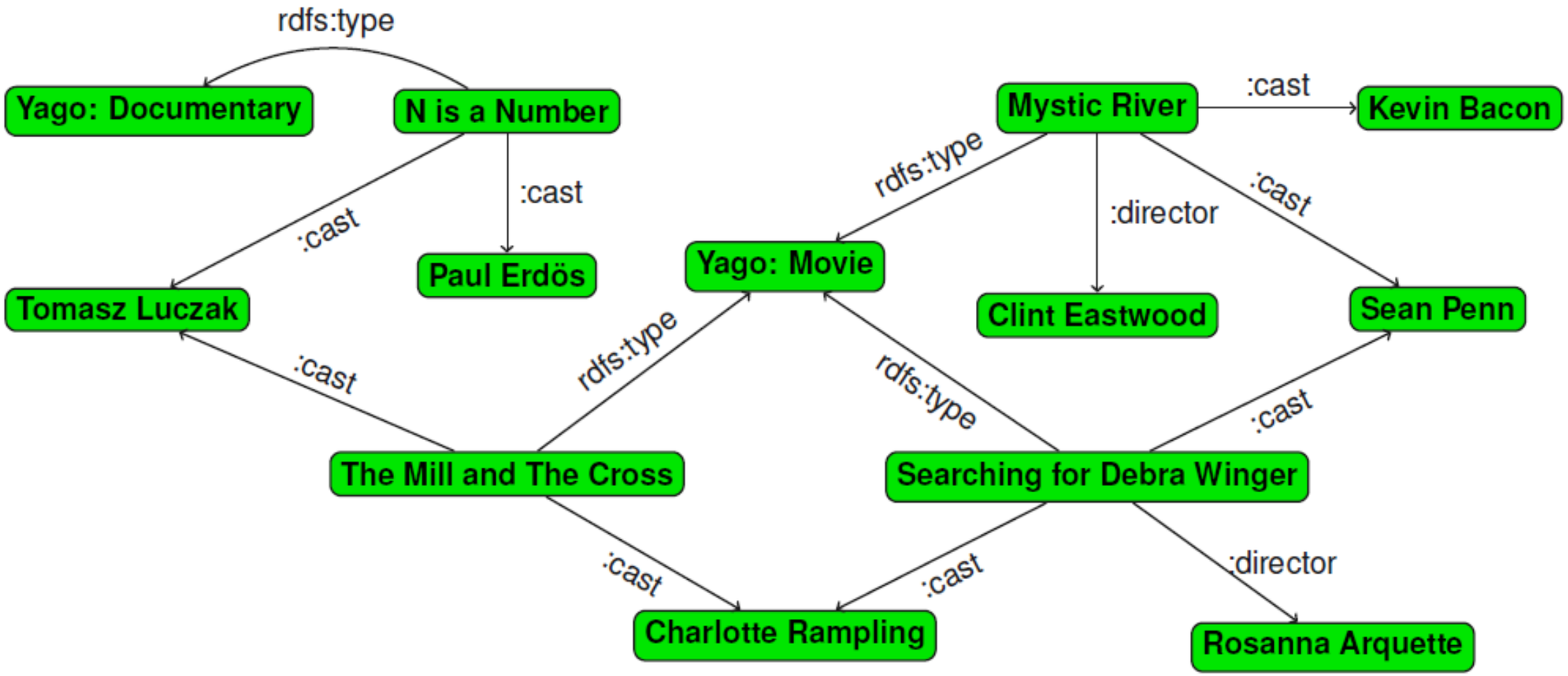
Resource Description Framework

- Especializadas en almacenar relaciones
- En general, almacenan sus datos como *property graphs*
- Algunos ejemplos: Neo4j, Virtuoso, Jena y Blazegraph



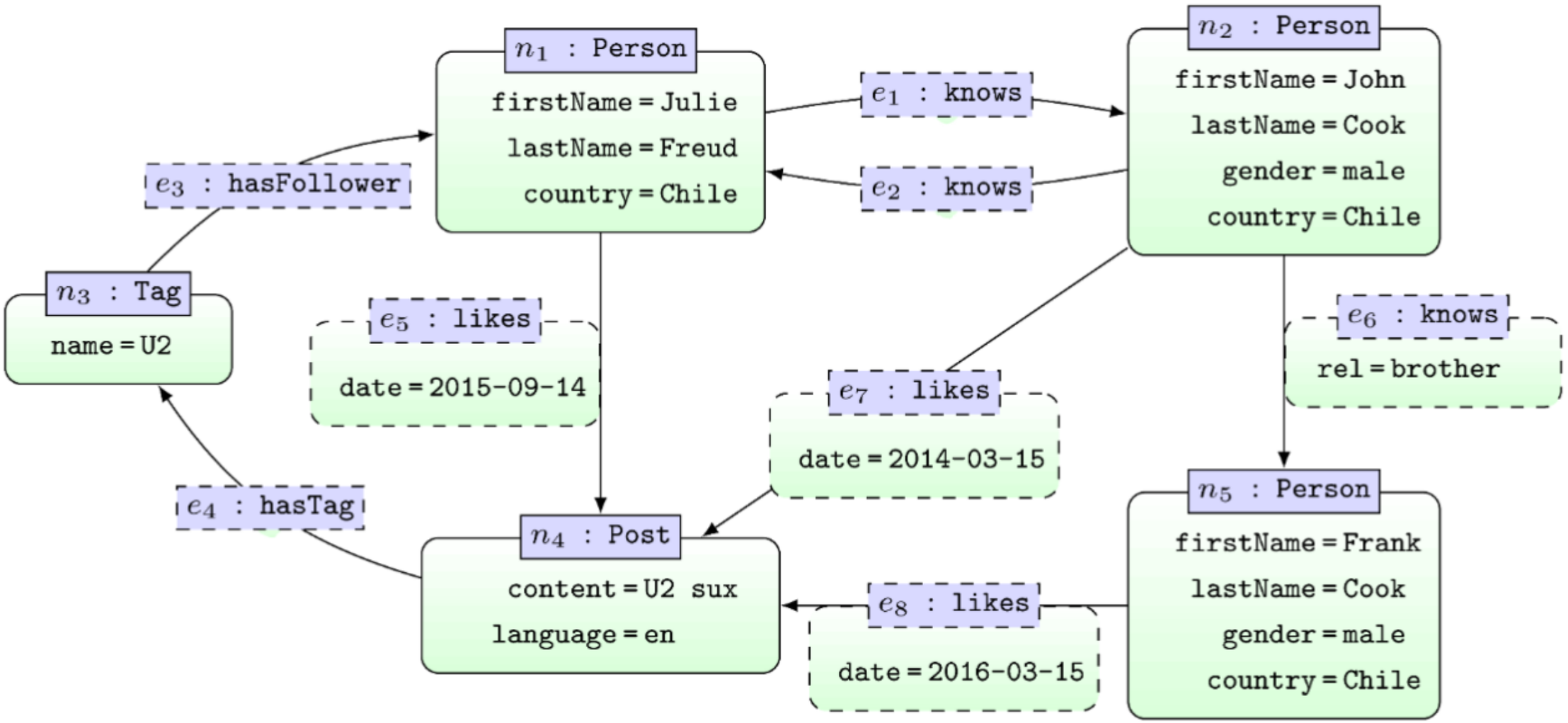
BD de grafos y RDF

Ejemplo: grafo RDF



BD de grafos y RDF

Ejemplo: property graph



BD orientadas a documentos

Definición

- Especializadas en documentos
- CouchDB, **MongoDB** (estas y otras BD almacenan sus datos en documentos JSON)
- JSON no es el único estándar de documentos (por ejemplo, también existe XML)



A continuación, la clase se centra en las **BD orientadas a documentos**, específicamente en **MongoDB**.

BD orientadas a documentos

Relación con los key-value stores

Son similares a los key-value stores:

- El valor es ahora un documento (JSON)
- Los documentos pueden agruparse (colecciones)
- Lenguaje de consultas mucho más poderoso




```
{
  "uid": 1,
  "name": "Adrian"
}
```

documento JSON

BD orientadas a documentos

Ejemplo: colección Usuarios

Usuarios	
Key	JSON

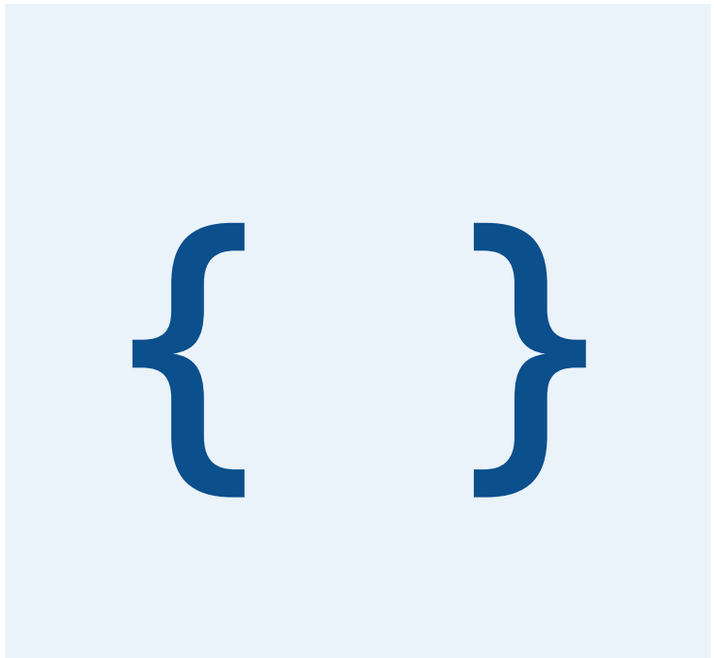
1	{
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

2	...
...	...

JSON

JavaScript Object Notation

- Su nombre proviene de JavaScript Object Notation
- Estándar de intercambio de datos semiestructurados / datos en la Web
- JSON se acopla muy bien a los lenguajes de programación



JSON – Ejemplo

Documento JSON


```
{
  "statuses": [
    {
      "id": 72545937362390630 4,
      "text": "@visitlondon: Have you been to any of these
               quirky London museums? https://t.co/tnrar8UttZ",
      "retweeted_status": {
        "metadata": {
          "result_type": "recent",
          "iso_language_code": "en"
        },
        "retweet_count": 239,
        "retweeted": false
      }
    }
  ]
}
```

JSON

Pares key-value y valores

La base son los pares key-value:

```
{
  "nombre": "Matías"
}
```

Los valores pueden ser:

- Números
- Strings (entre comillas)
- Valores booleanos
- Arreglos (por definir)
- Objetos (por definir)
- null

JSON – Sintaxis

Objetos

Los objetos se escriben entre **{ }** y contienen una cantidad arbitraria de pares key-value.

```
{
  "nombre": "Matías", "apellido": "Jünemann"
}
```

Cada par se separa del siguiente mediante una coma.

JSON – Sintaxis

Arreglos

```
{
  "profesores": [
    {"nombre": "Juan", "apellido": "Reutter"},
    {"nombre": "Cristian", "apellido": "Riveros"},
    {"nombre": "Marcelo", "apellido": "Arenas"}
  ]
}
```

Los arreglos se escriben entre **[ ]** y contienen valores.

JSON vs. SQL

Comparación

SQL

- Esquema de datos
- Lenguajes de consulta independientes del código

JSON

- Más flexible: no es necesario respetar un esquema
- Más tipos de datos (como arreglos)
- Legible por humanos (human-readable)

Ejemplo

Almacenar varios roles por persona

ID	Persona	rol
1	Juan Soto	Paciente y Médico
2	Pedro Pérez	Paciente

SQL →

ID	Persona
1	Juan Soto
2	Pedro Pérez

1 — N

#	ID	rol
1	1	Paciente
2	1	Médico
3	2	Paciente

MongoDB

colección

documento

```
ID: 1
Nombre: "Juan Soto"
Rol: ["Paciente",
      "Médico"]
```

documento

```
ID: 2
Nombre: "Pedro Pérez"
Rol: "Paciente"
```

BD de documentos: ¿para qué?

Propósito

Especializadas en documentos: almacenan una gran cantidad de documentos JSON.

Libros

Un documento JSON por libro

Personas

Un documento JSON por persona

Esta organización es altamente jerárquica.

BD de documentos

Fortalezas

Lo que realizan bien:

- Obtener un libro o una persona en particular
- Cruce de información **simple**
- Muy útiles para desplegar información en la Web

Además, pueden utilizarse como **caché** de una BD relacional.

Caché de BD SQL

Esquema relacional de ejemplo

Students

StudentID	Nombre	Carrera
1	Alice Cooper	Computación
2	David Bowie	Todas
3	Charly García	Ingeniería Civil
…	…	…

Courses

courseID	name	year
IIC2413	Databases	2020
IMT3830	Game Theory	2020
…	…	…

Takes

courseID	StudentID
IIC2413	1
IIC2413	2
IMT3830	2
…	…

Takes relaciona alumnos y cursos (relación N:M).

Caché de BD SQL

Lista de alumnos por curso

Para obtener la lista de alumnos por curso:

SQL

Es necesario realizar un join

`Students ⋈ Takes ⋈ Courses`

BD de documentos

Esta información se prepara de antemano

`Colección "Courses"`

Caché de BD SQL

Colección "Courses"

```
{
  "courseID": IIC2413,
  "name": "Databases",
  "year": 2020,
  "students": [
    {
      "studentID": 1,
      "name": "Alice Cooper"
    },
    {
      "studentID": 2,
      "name": "David Bowie"
    },
    ...
  ]
}
```

```
{
  "courseID": IMT3830,
  "name": "Game Theory",
  "year": 2020,
  "students": [
    {
      "studentID": 2,
      "name": "David Bowie"
    },
    {
      "studentID": 3,
      "name": "Charly García"
    },
    ...
  ]
}
```

BD de documentos

Fortalezas en este caso

Lo que realizan bien en este caso:

- Obtener la lista de alumnos de un curso
- Obtener los nombres de todos los cursos
- Obtener todos los cursos tomados por “David”

BD de documentos

Debilidades

Lo que realizan mal:

Datos cambiantes

Manejo de información que cambia mucho

Cruces complejos

Cruce de información no trivial

BD de documentos

Cruce de información

Colección "Courses": todos los alumnos que toman IIC2413 y IMT3830

```
{
  "courseID": IIC2413,
  "name": "Databases",
  "year": 2020,
  "students": [
    {
      "studentID": 1,
      "name": "Alice Cooper"
    },
    {
      "studentID": 2,
      "name": "David Bowie"
    },
    ...
  ]
}
```

```
{
  "courseID": IMT3830,
  "name": "Game Theory",
  "year": 2020,
  "students": [
    {
      "studentID": 2,
      "name": "David Bowie"
    },
    {
      "studentID": 3,
      "name": "Charly García"
    },
    ...
  ]
}
```

BD de documentos

Cruce de información: nested loop join

Colección "Courses": todos los alumnos que toman IIC2413 y IMT3830

Efectivamente, es necesario realizar un nested loop join:

- Iterar por todos los alumnos de IMT3830
- Iterar por todos los alumnos de IIC2413
- Verificar si hacen join

MongoDB soporta JavaScript y Python:

- Es posible realizarlo, pero no es una solución elegante

```
for a in imt3830["students"]:
  for b in iic2413["students"]:
    if a["studentID"] ==
       b["studentID"]:
      resultado.append(a)
```

Esquema del nested loop join (Python)

En resumen

BD de documentos vs. BD SQL

BD de documentos

- Útiles para el despliegue de información estática
- Búsquedas simples
- Cruces muy sencillos

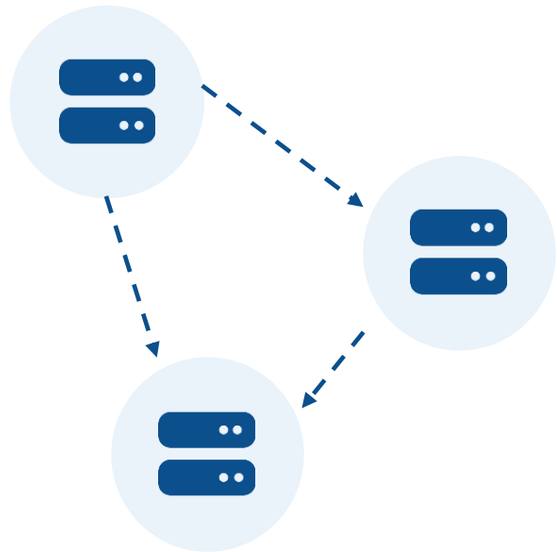
BD SQL

- La información cambia mucho
- Se requieren cruces con frecuencia
- Se necesita ACID

BD de documentos y BASE

Consistencia eventual

- Distintas aplicaciones en una misma base de datos acceden a distintos documentos al mismo tiempo
- En general, diseñadas para montar varias instancias que (en teoría) contienen la misma información
- Propagan las actualizaciones (updates) en forma descoordinada



Proveen **consistencia eventual**: con el tiempo, todos los nodos del sistema llegarán a un estado consistente, lo que permite alta disponibilidad y tolerancia a fallos, a costa de no tener consistencia inmediata.

Consistencia eventual

Problemas

No obstante, la consistencia eventual puede generar problemas: si dos aplicaciones intentan acceder al mismo documento en MongoDB, pueden obtener versiones diferentes de dicho documento.



La réplica 2 aún no recibe la actualización.



MongoDB

Bases de datos orientadas a documentos

MongoDB

Características

Base de datos NoSQL orientada a documentos que almacena datos en formato JSON (BSON). Sus características son:

Flexibilidad de esquema

Permite almacenar documentos con diferentes estructuras sin migraciones complejas.

Escalabilidad horizontal

Distribuye los datos en múltiples servidores para mejorar el rendimiento y la capacidad.

Alto rendimiento

Optimizado para operaciones rápidas de lectura y escritura.

Replicación y alta disponibilidad

Asegura la disponibilidad y la recuperación ante fallos mediante réplicas.

MongoDB

Documentos

Usuarios	
Key	JSON

60bfd90e002ce228636e506b	{
	"_id": ObjectId('60bfd90e002ce228636e506b'),
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

60bfd90e002ce228636e5215	...
...	...

MongoDB

Colecciones

Colección: una agrupación de documentos similares

Usuarios	
Key	JSON

60bfd90e002ce228636e506b	{
	"_id": ObjectId('60bfd90e002ce228636e506b'),
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

60bfd90e002ce228636e5215	...
...	...

MongoDB

Bases de datos

Base de datos: contiene colecciones relacionadas

Base de datos

Usuarios	
Key	JSON
…	…

Mensajes	
Key	JSON
…	…

Likes	
Key	JSON
…	…

MongoDB

Servidor

Un **servidor** contiene varias bases de datos



Consultando a MongoDB

Comandos básicos

Comando	Descripción
`show dbs`	Muestra las bases de datos disponibles
`use dbName`	Selecciona la base de datos dbName
`show collections`	Muestra las colecciones de la base de datos
`db.colName.find()`	Todos los documentos de la colección colName
`db.colName.find().pretty()`	Muestra los datos con un mejor formato
`db.colName.find({"name": "Adrian"})`	Selección
`db.colName.find({"age": {$gte: 23}})`	Selección
`db.colName.find({"age": {$gte: 23}},` `{"name": 1})`	Proyección



Full-text search & retrieval

Búsqueda y recuperación de texto

¿Cómo buscar texto?

Búsqueda de texto



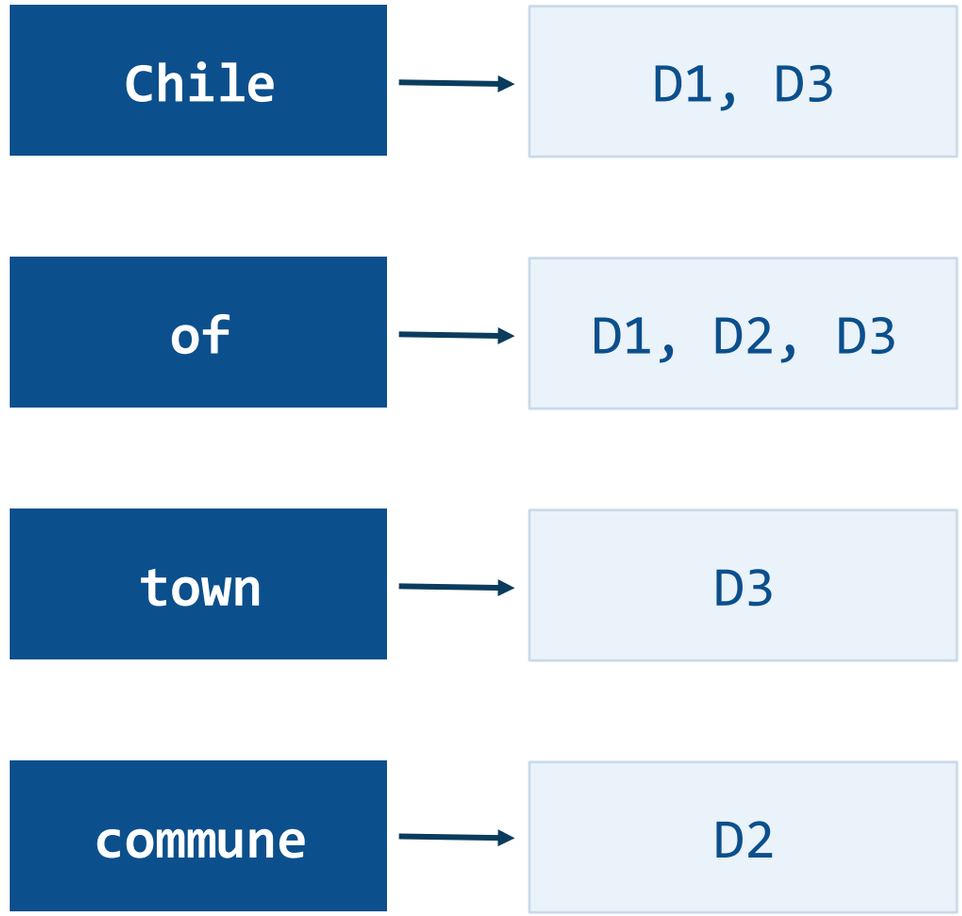
Al buscar “Chilean Mammal”:

- El sistema encuentra todos los documentos que contienen *Chilean* y *Mammal*
- Pero, ¿cómo se ordenan los resultados? Puede ser problemático en grandes bases de datos

Índices invertidos

Definición

- Para hacer la búsqueda eficiente se utilizan **índices invertidos**
- Para cada palabra del universo de documentos, se guardan punteros que indican dónde están los documentos



Índices invertidos

Ejemplo

Palabra	Documento 1	Documento 2	Documento 3	…
Chile	1	0	1	
of	1	1	1	
town	0	0	1	
commune	0	1	0	
…				

1: la palabra aparece en el documento · 0: no aparece

Text search en MongoDB

Índice de texto

Un índice especial permite la búsqueda rápida de texto:

```
db.colName.createIndex({"attributeName": "text"})

db.users.find({$text: {$search: "Delantero de Cobreloa"}})
```

Ver más en: https://www.youtube.com/watch?v=vR97-4UG7x0

Vectores dispersos

Sparse vectors

- Los documentos se representan como **vectores en un espacio de alta dimensión**, donde cada término del vocabulario ocupa una dimensión única
- Los **vectores dispersos** (sparse vectors) tienen la mayoría de sus posiciones en cero, ya que un documento solo utiliza una fracción pequeña del vocabulario total. Esto optimiza radicalmente el almacenamiento y el procesamiento



Vector de un documento: solo unas pocas posiciones son distintas de cero

¿Por qué usar vectores dispersos?

Ventajas

Coincidencia precisa

Captura términos exactos.

Resultados predecibles

El comportamiento es transparente y auditable.

Almacenamiento eficiente

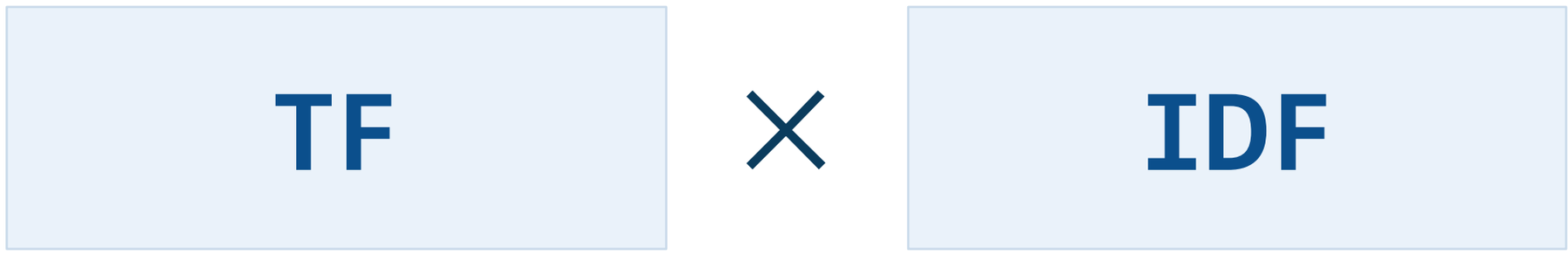
Solo se almacenan los valores distintos de cero, como pares índice-valor.

TF-IDF

Definición

Frecuencia del término – frecuencia inversa de los documentos

Mide la importancia de una palabra en un documento en comparación con un conjunto de documentos.



TF-IDF

Principios

↑ Principio 1

El puntaje es **proporcional** a la cantidad de veces que aparece la palabra en el documento

↓ Principio 2

El puntaje es **inversamente proporcional** a la cantidad de documentos en los que aparece la palabra

TF-IDF

Fórmula

Term Frequency:

$F_D(t)$ = número de veces que aparece t en D

Inverse Document Frequency:

```
IDF(t) = log(número de documentos /
             número de documentos en los que aparece t)
```

Entonces:

$$\text{TF-IDF} = F_D(t) \cdot \text{IDF}(t)$$

TF-IDF

Ejemplo

D1	“Ojo por ojo, diente por diente”
D2	“Ojo por ojo, y el mundo acabará ciego”
D3	“Si luchas contra el mundo, ponte del lado del mundo”

A continuación, se calcula el TF-IDF de “ojo” y “mundo” para cada documento.

TF-IDF: ejemplo

Paso 1: calcular TF

El TF se calcula como el número de veces que aparece un término en un documento, dividido por el número total de términos del documento.

Documento	Total de palabras	TF(“ojo”)	TF(“mundo”)
D1: “Ojo por ojo, diente por diente”	6	`2/6 = 0.333`	`0/6 = 0`
D2: “Ojo por ojo, y el mundo acabará ciego”	8	`2/8 = 0.25`	`1/8 = 0.125`
D3: “Si luchas contra el mundo, ponte del lado del mundo”	10	`0/10 = 0`	`2/10 = 0.2`

TF-IDF: ejemplo

Paso 2: calcular IDF

El IDF se calcula como el logaritmo del número total de documentos dividido por el número de documentos que contienen el término.

3

documentos en total

Término	Documentos que lo contienen	IDF
“ojo”	2 (D1 y D2)	`log(3/2) = 0.176`
“mundo”	2 (D2 y D3)	`log(3/2) = 0.176`

TF-IDF: ejemplo

Paso 3: calcular TF-IDF

El TF-IDF se calcula multiplicando el TF por el IDF.

Documento	TF-IDF para “ojo”	TF-IDF para “mundo”
D1	`0.333 · 0.176 = 0.059`	`0 · 0.176 = 0`
D2	`0.25 · 0.176 = 0.044`	`0.125 · 0.176 = 0.022`
D3	`0 · 0.176 = 0`	`0.2 · 0.176 = 0.035`

TF-IDF

En la práctica



Existen distintas funciones para TF e IDF



Generalmente se incorporan funciones de stemming y stop words para el procesamiento de lenguaje natural



Cada compañía tiene su propia receta, que depende además del idioma

TF-IDF

Búsqueda de documentos

- Se genera una matriz en la que las dimensiones son las palabras y los documentos
- Cada celda indica el TF-IDF de la palabra en cada documento
- Cuando un usuario busca una frase, se genera un vector y se retornan los documentos con los vectores más similares

	D1	D2	D3
ojo	0.059	0.044	0
mundo	0	0.022	0.035

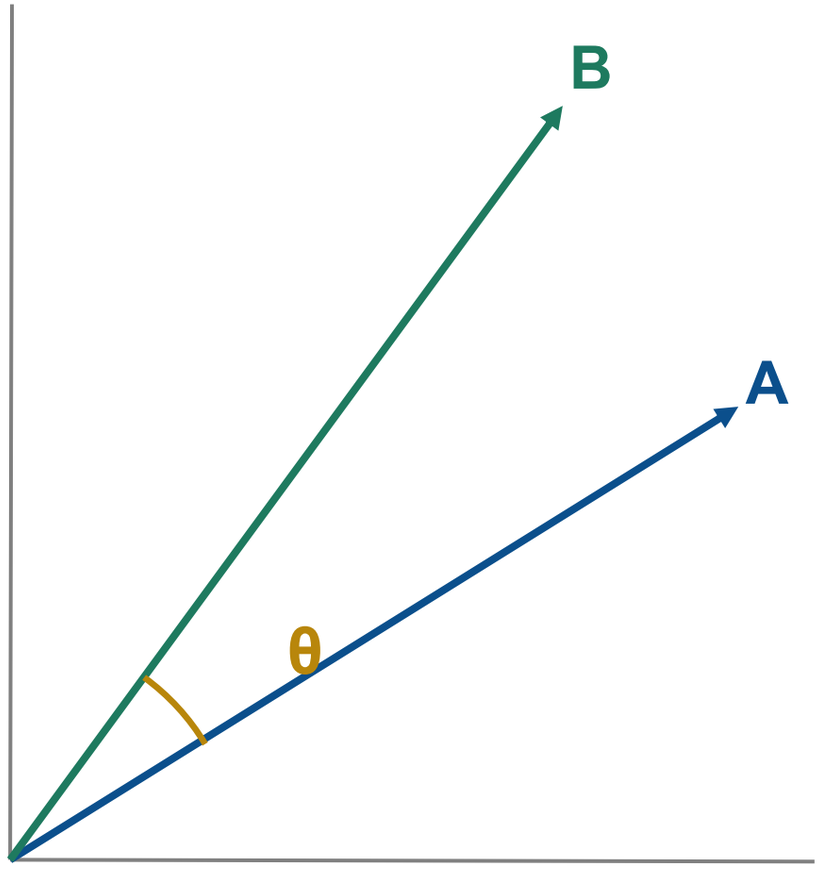
Matriz TF-IDF del ejemplo anterior

Medición de similitud entre documentos

Búsqueda de documentos

- **Similitud del coseno:** mide el ángulo entre dos vectores en el espacio de términos
- Cuanto **menor el ángulo** entre dos vectores, mayor es su similitud

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$





MapReduce

Computación distribuida

MapReduce

Definición

Algoritmo eficiente para computación paralela/distribuida

Paradigma de programación: mezcla de datos con controlador

Sacrificios para acelerar la computación

Computación distribuida

Motivación

Datos tan grandes que no caben en un computador

Repartidos en muchos servidores

Cada servidor no conoce lo que tiene el resto

No es posible darse el lujo de comunicar todos los datos

MapReduce

Problema de ejemplo

- Ejemplo: contar cuántas veces ocurre cada palabra en un texto T
- ¿Cómo hacer esto en un archivo de 100 **petabytes***?

100 PB

de texto

*100 petabytes = 100.000.000 gigabytes

MapReduce

Etapas

Map

Recibe datos y genera pares key–value

Shuffle

Transfiere los datos desde los mappers a los reducers

Reduce

Recibe los pares con la misma key y los agrega

MapReduce – Arquitectura

Mappers y reducers

Mappers

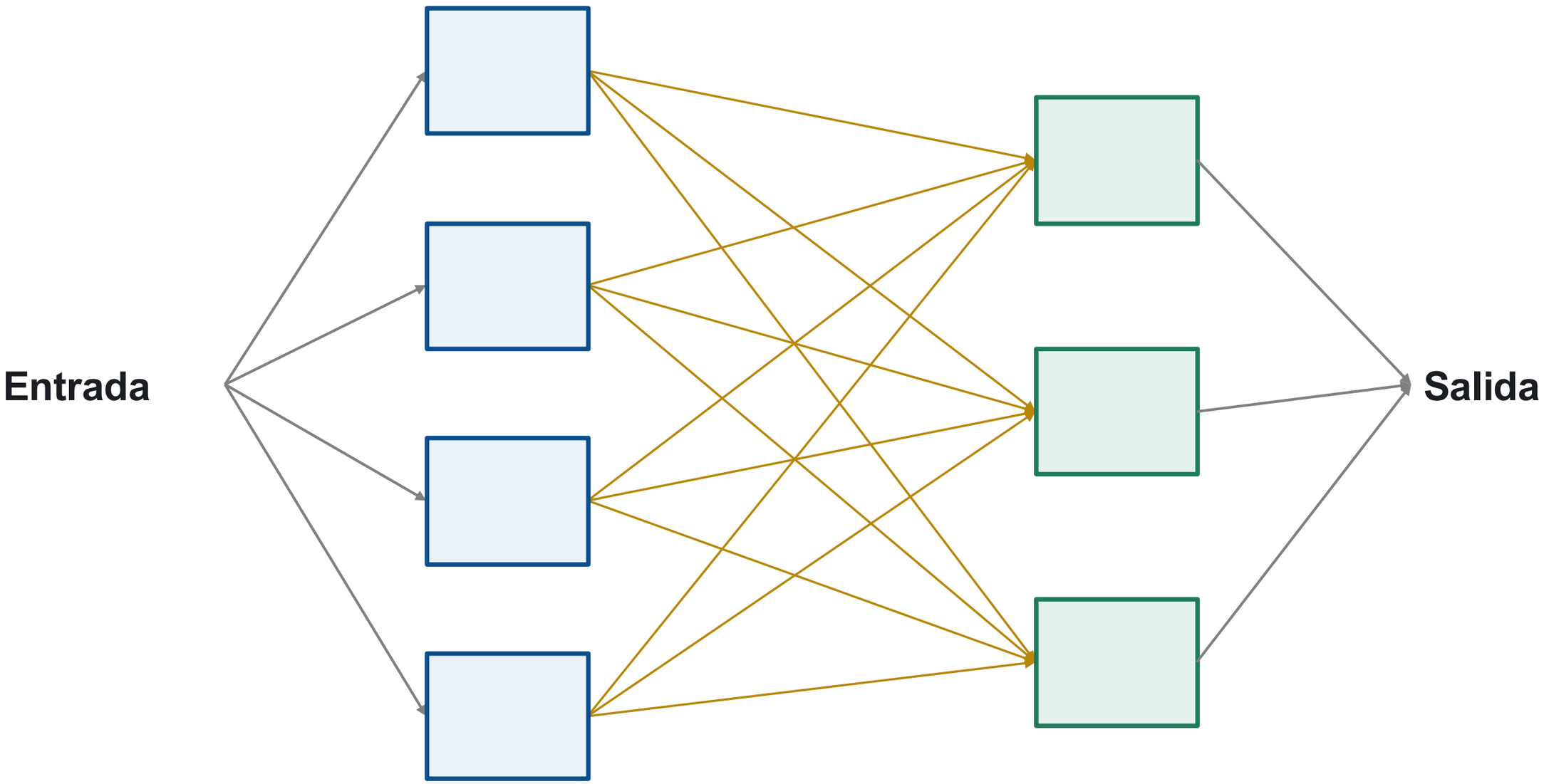
- Nodos encargados de realizar Map
- Reciben parte del documento y lo envían a los reducers (a través del **shuffle**)

Reducers

- Nodos encargados de realizar Reduce
- Reciben los resultados de Map y los agregan
- El output es la unión de cada reducer

MapReduce

Arquitectura general



MapReduce – Ejemplo

Conteo de palabras

¿Cuántas veces ocurre cada palabra en un archivo de texto grande? Para saberlo, se definen:

Map

- Recibe un fragmento de texto
- Por cada palabra, emite el par (palabra, número de ocurrencias)

Reduce

- Cada reducer recibe todos los pares asociados a la misma palabra
- Junta todos estos pares y suma las ocurrencias

MapReduce – Ejemplo

Input

Texto T

```
hola que hola
año zzz hola
que zzz que
```

Máquina map1

Máquina map2

Máquina map3

Máquina reduce1 · palabras A–M

Máquina reduce2 · palabras N–Z

MapReduce – Ejemplo

Separar input

Texto T

```
hola que hola
año zzz hola
que zzz que
```

Máquina map1

Máquina map2

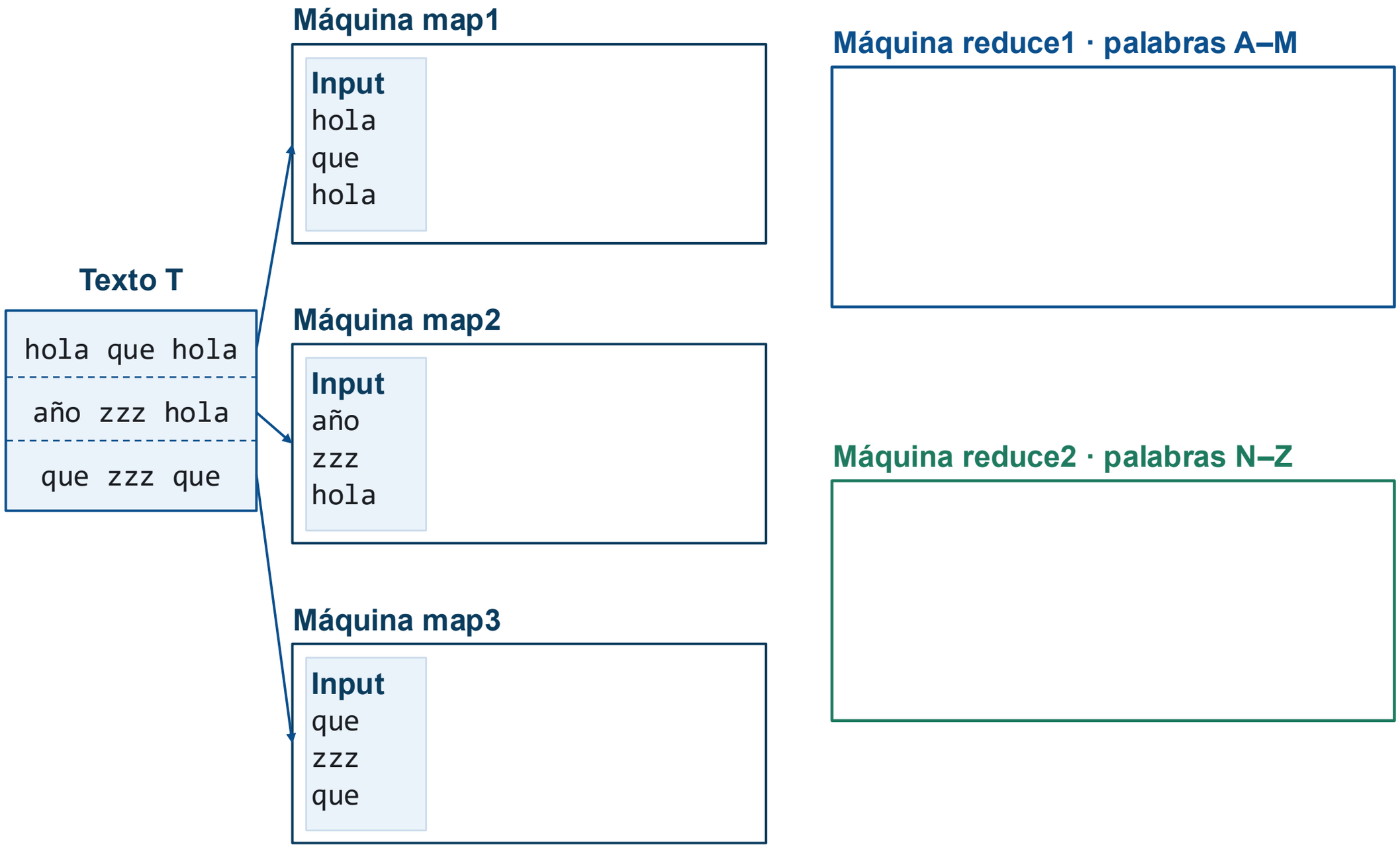
Máquina map3

Máquina reduce1 · palabras A–M

Máquina reduce2 · palabras N–Z

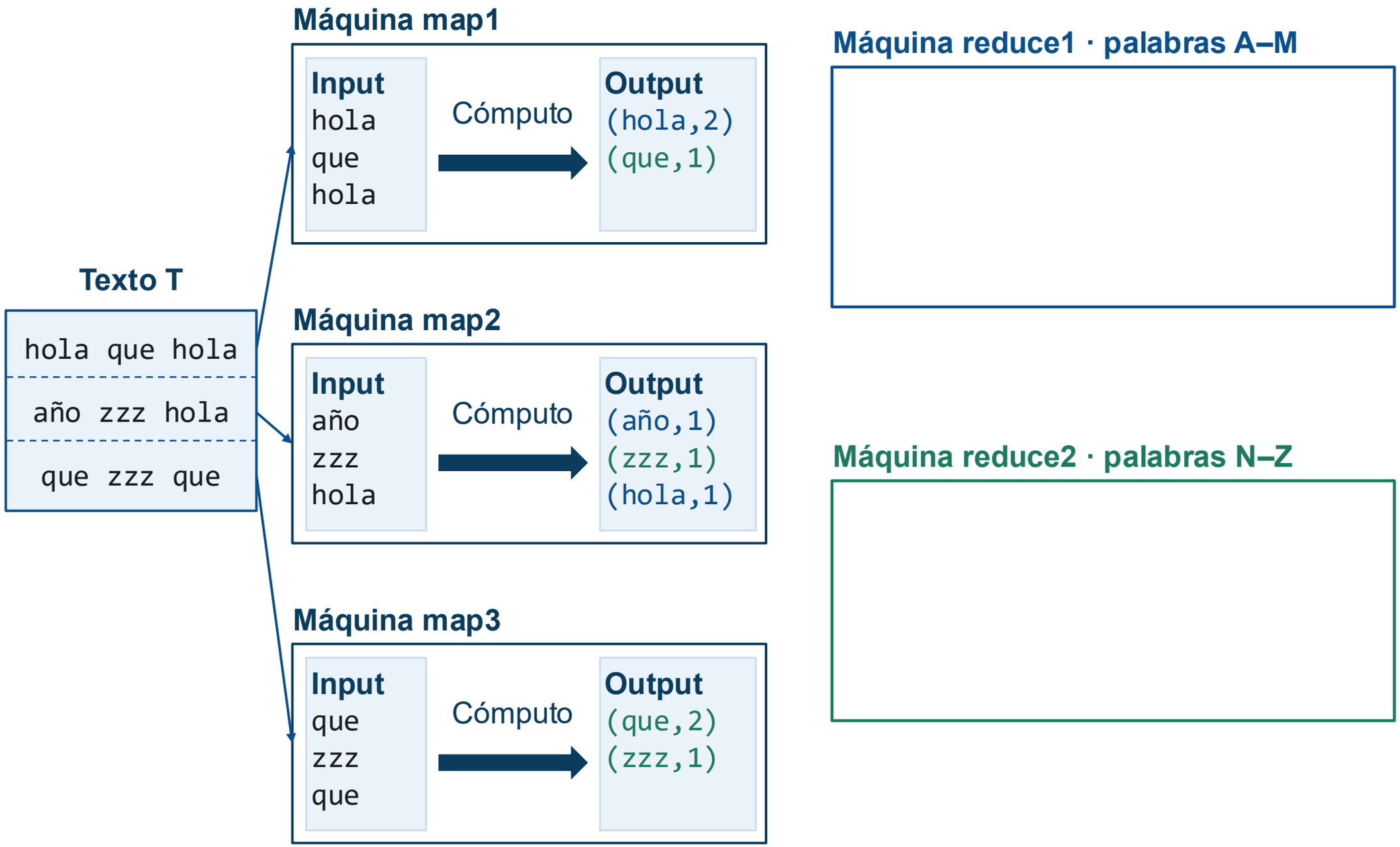
MapReduce – Ejemplo

Separar input



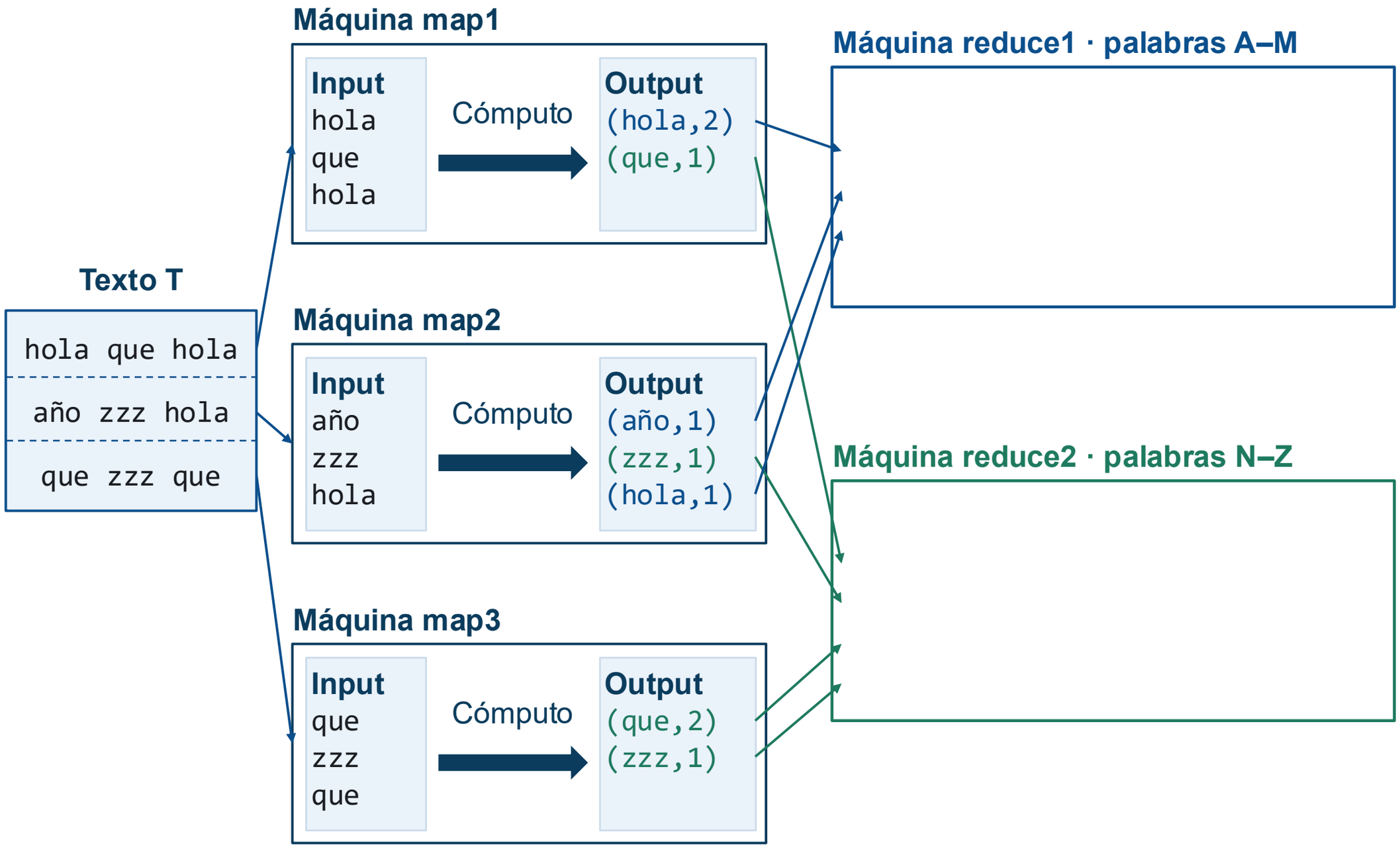
MapReduce – Ejemplo

Map



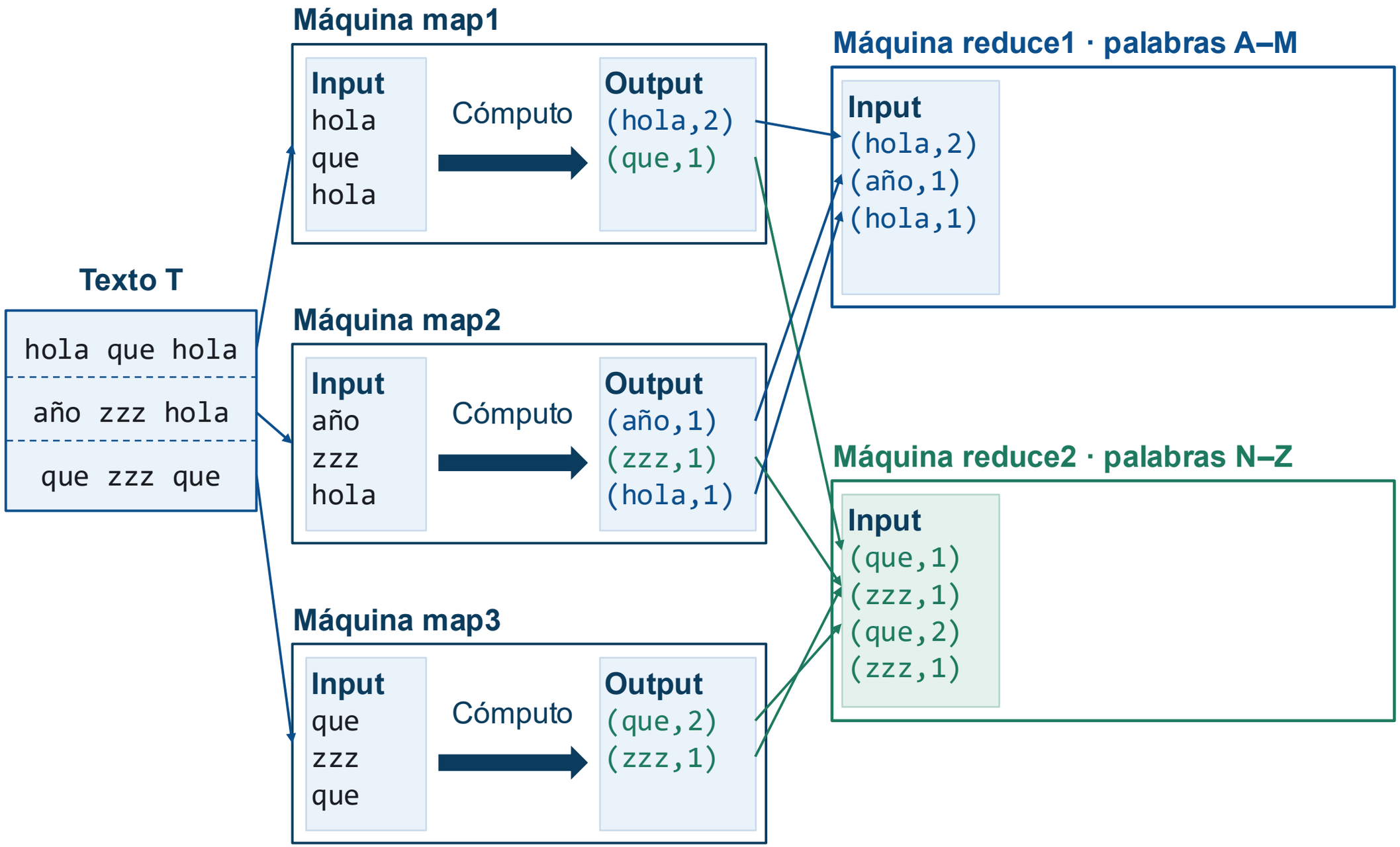
MapReduce – Ejemplo

Shuffle



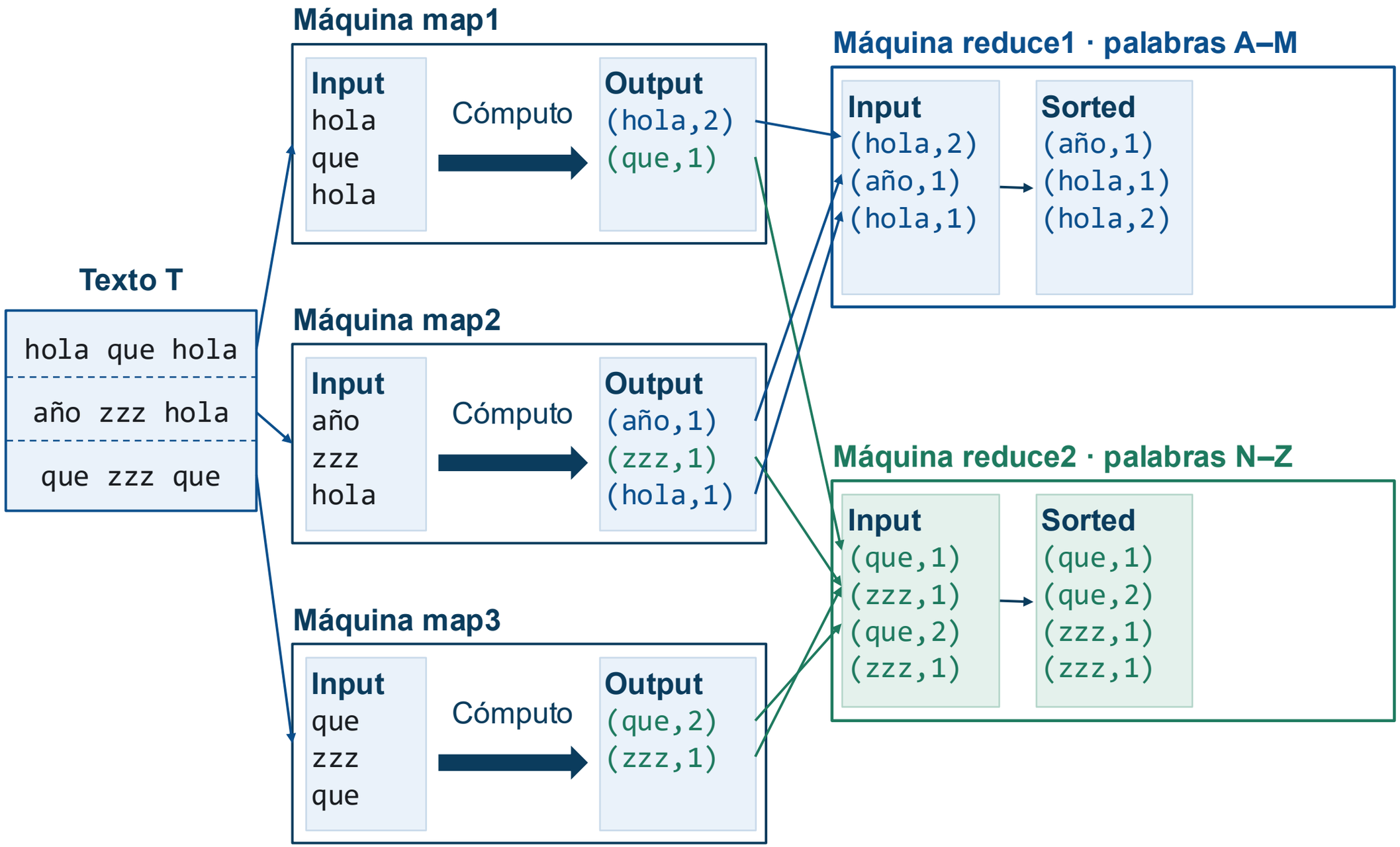
MapReduce – Ejemplo

Shuffle



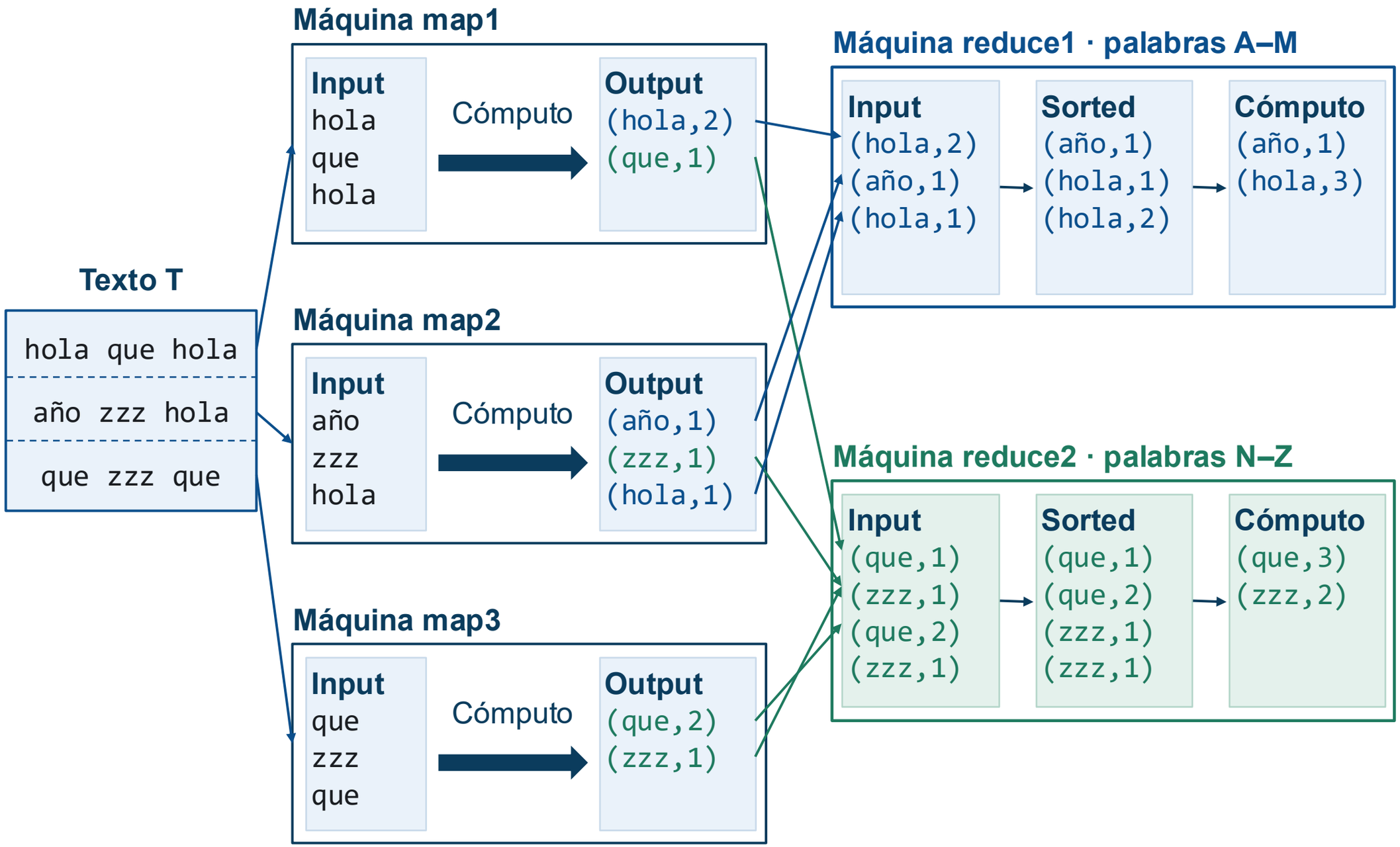
MapReduce – Ejemplo

Reduce



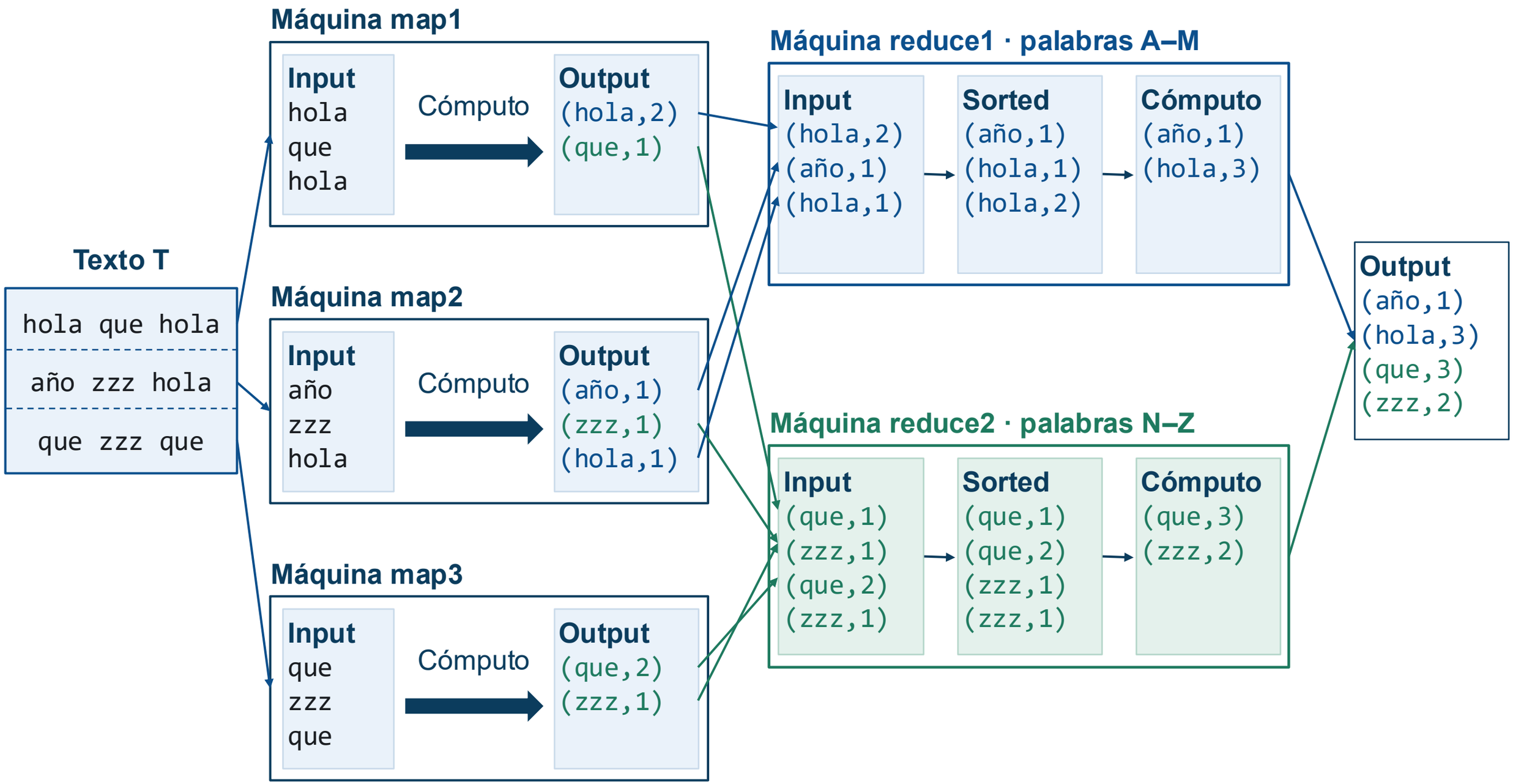
MapReduce – Ejemplo

Reduce



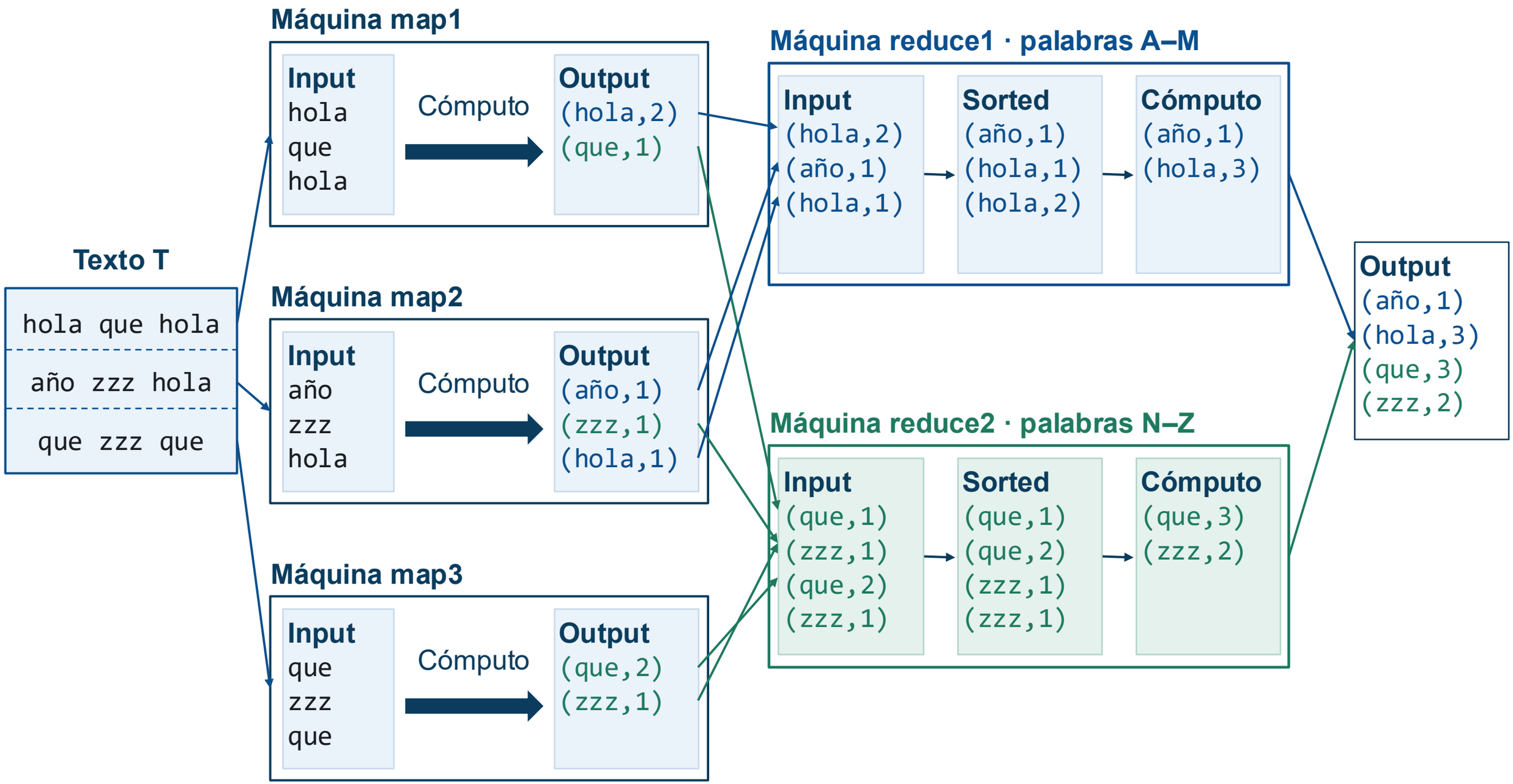
MapReduce – Ejemplo

Output



MapReduce – Ejemplo

Resumen del proceso



MapReduce – Ejemplo: join

Planteamiento

¿Cómo se realiza un join con MapReduce?

- **Modelo:** un archivo con el nombre de la tabla y sus tuplas
- **Map:** se agrupa por el atributo sobre el que se hace el join
- **Reduce:** se realiza el producto cruz para las tablas distintas

$$R(A, B) \bowtie S(B, C)$$

MapReduce – Ejemplo: join

Input

R

A	B
1	1
1	3
3	2
3	3

S

B	C
2	4
2	7
3	8
3	9

Máquina map1

Máquina map2

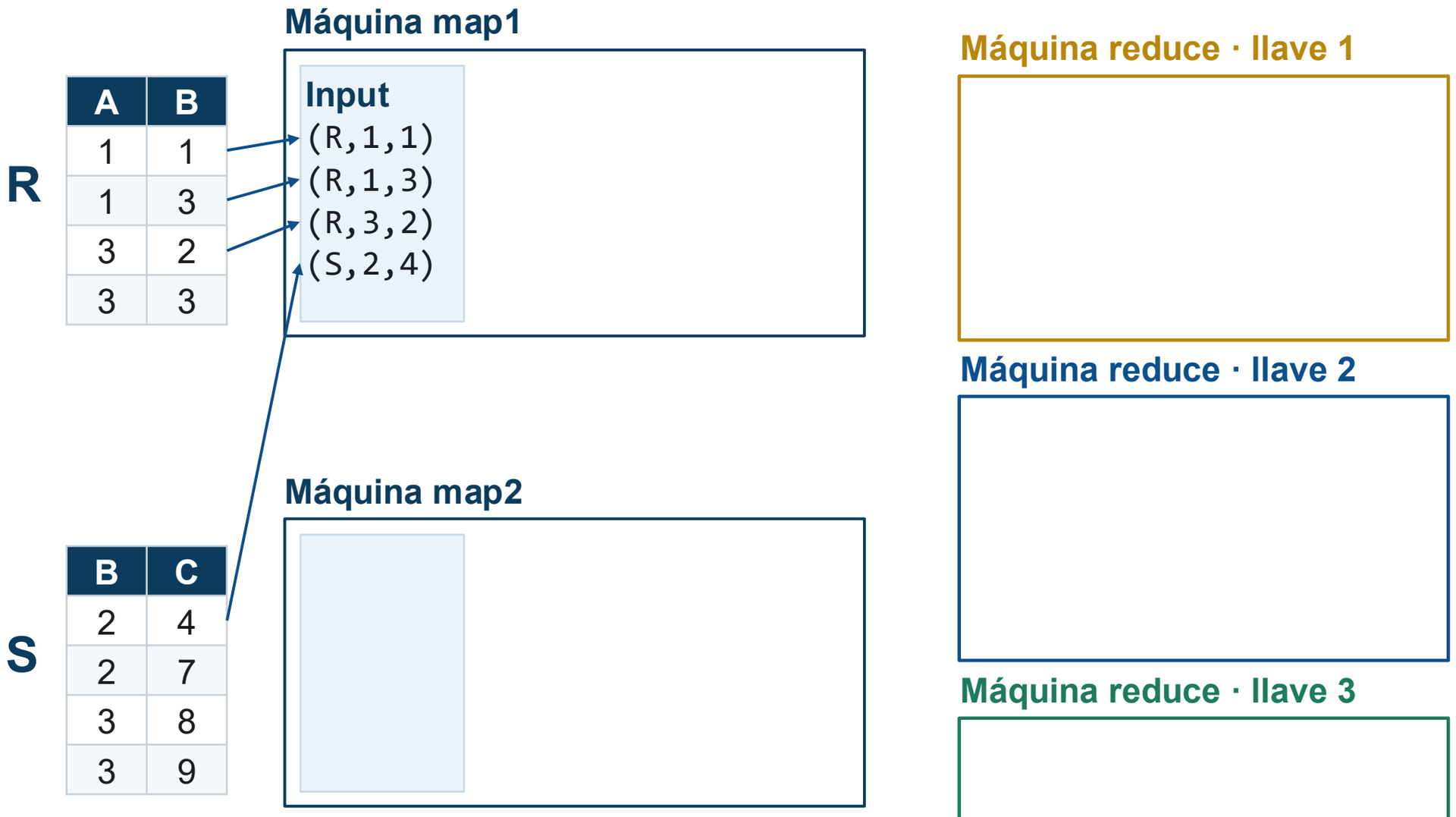
Máquina reduce · llave 1

Máquina reduce · llave 2

Máquina reduce · llave 3

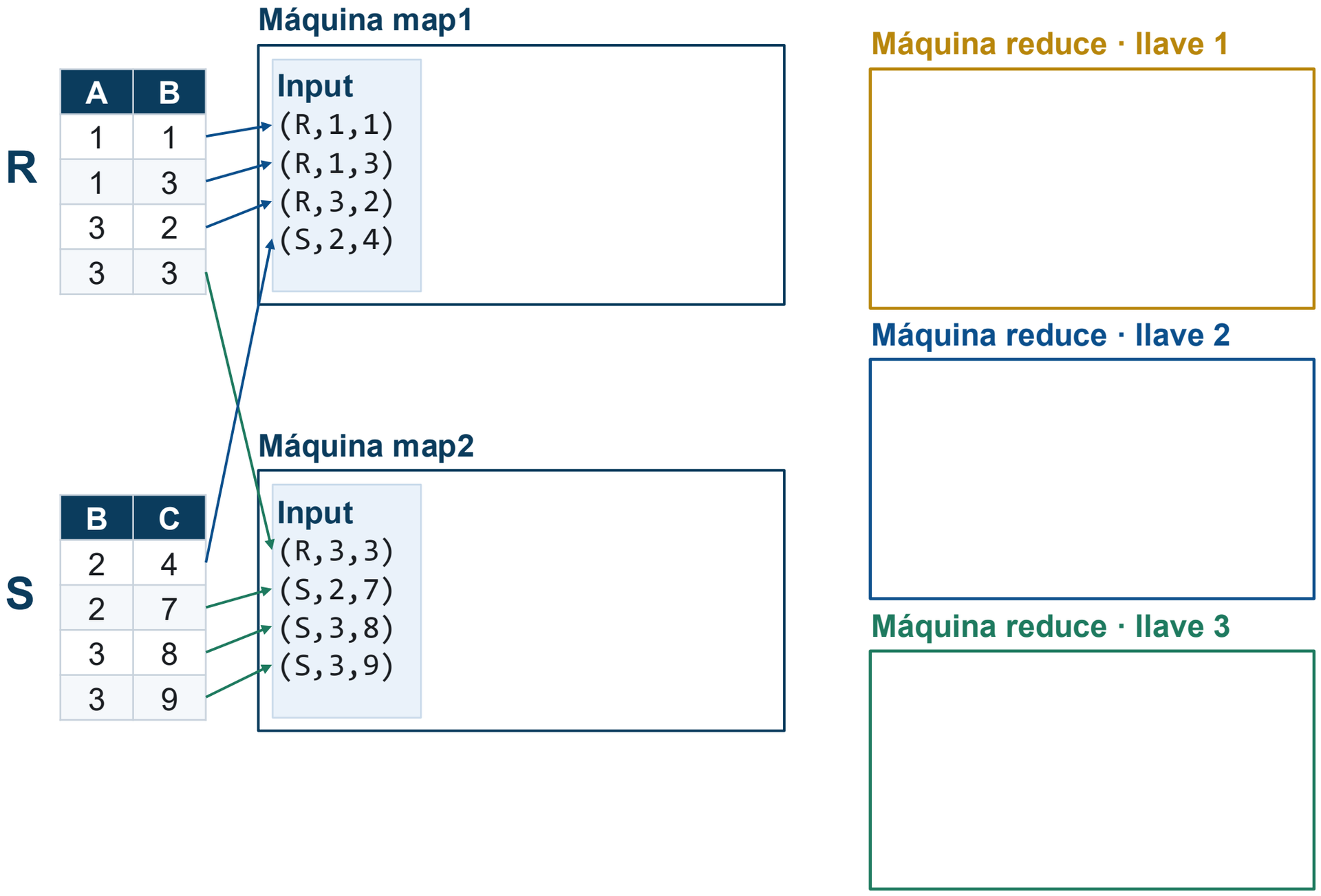
MapReduce – Ejemplo: join

Map



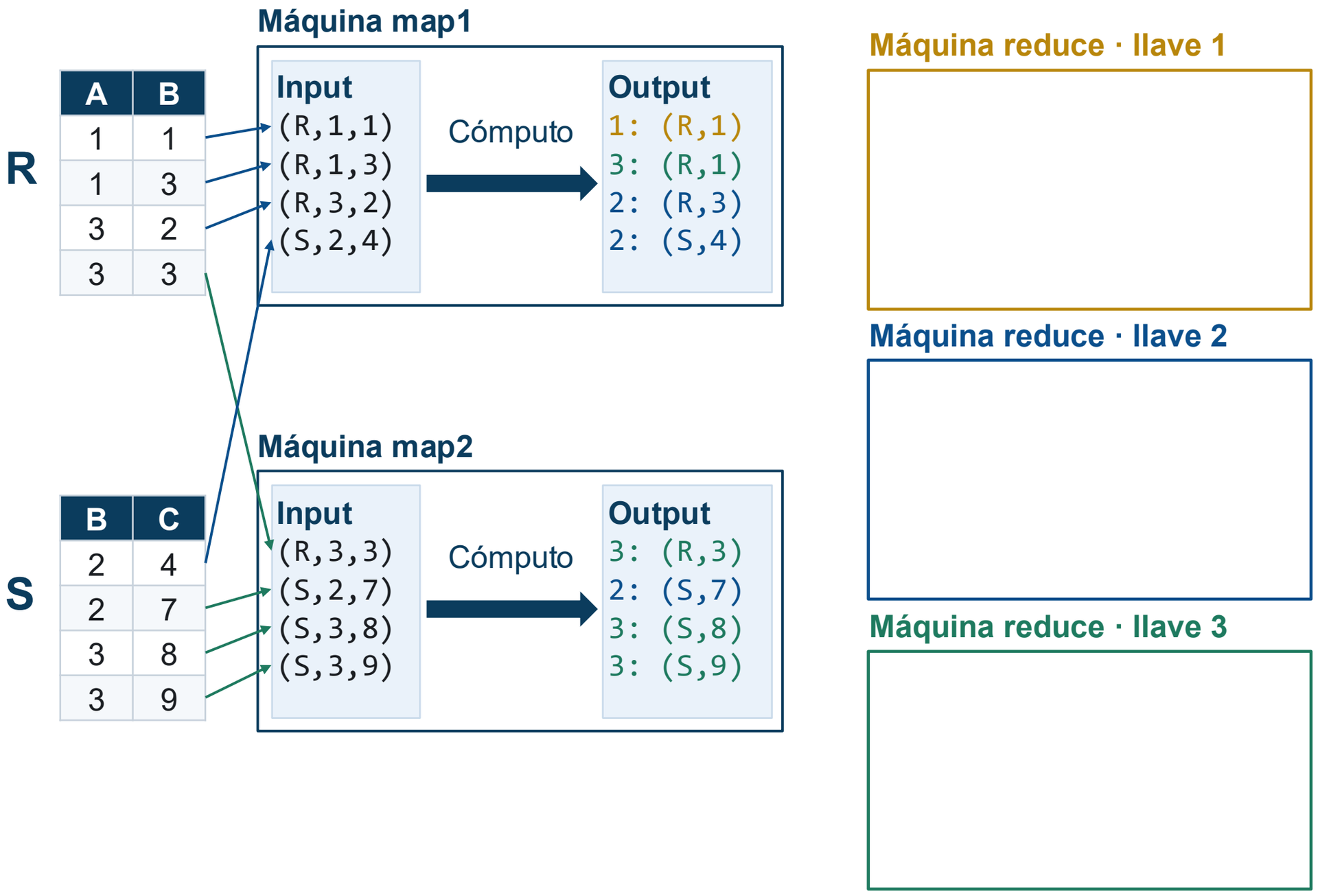
MapReduce – Ejemplo: join

Map



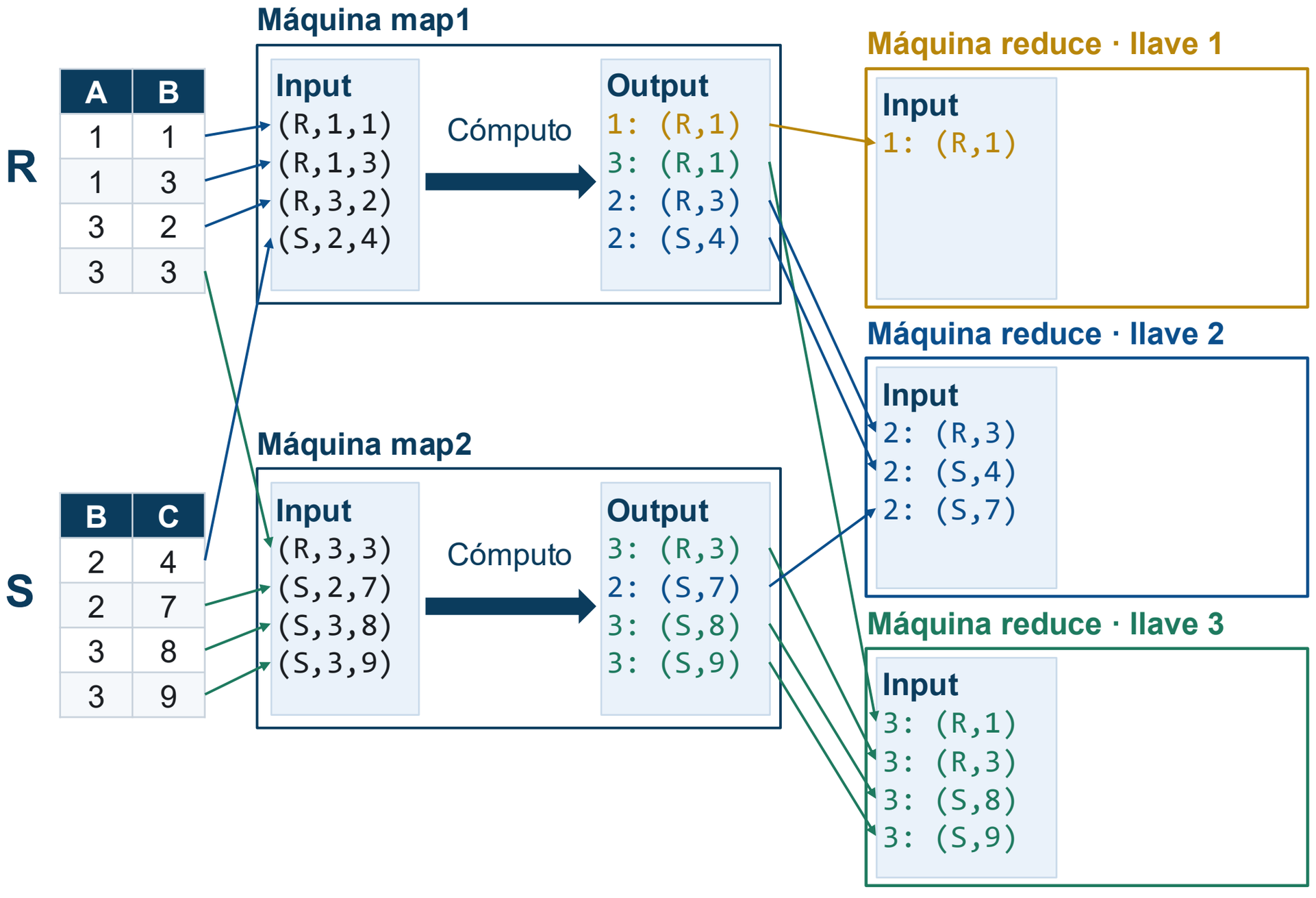
MapReduce – Ejemplo: join

Map



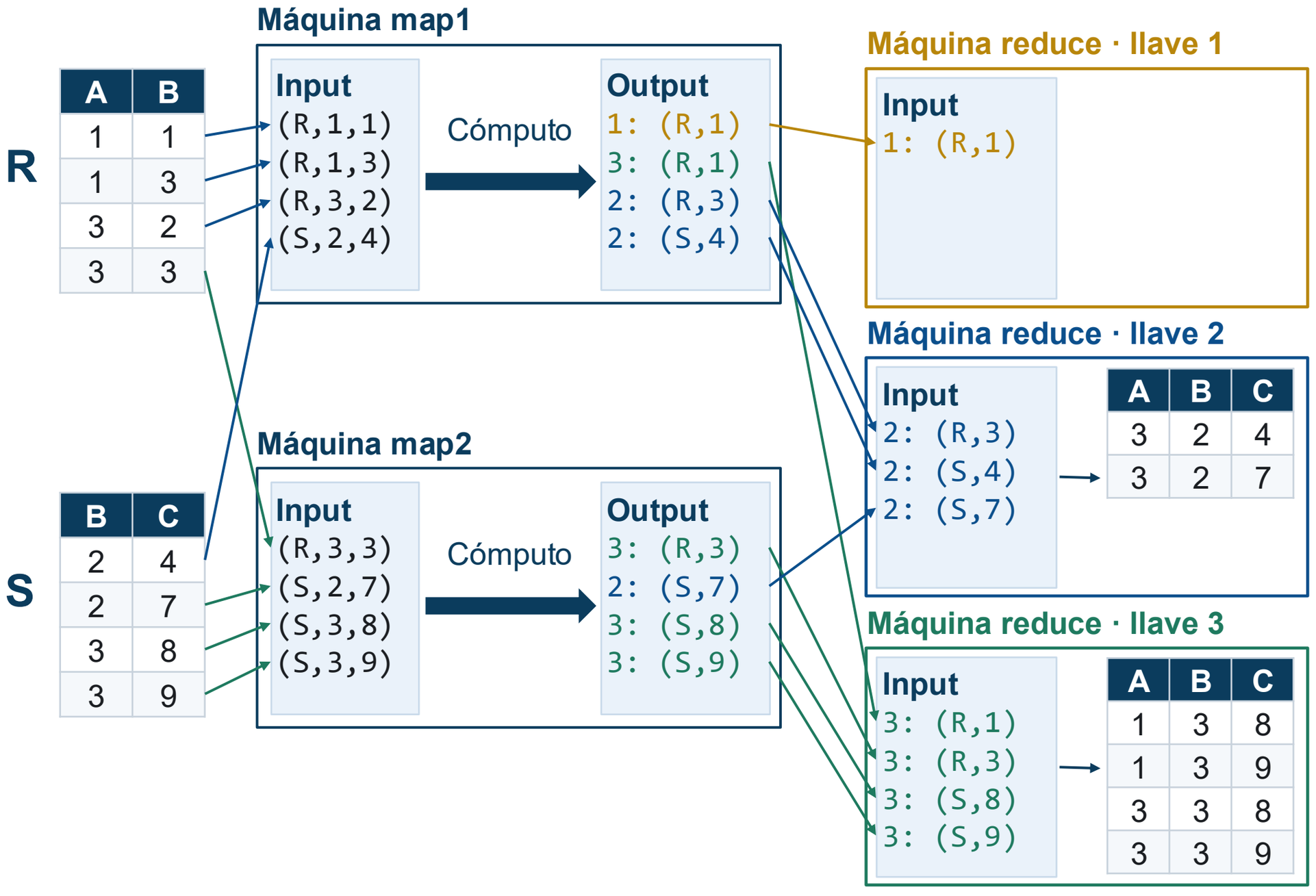
MapReduce – Ejemplo: join

Shuffle



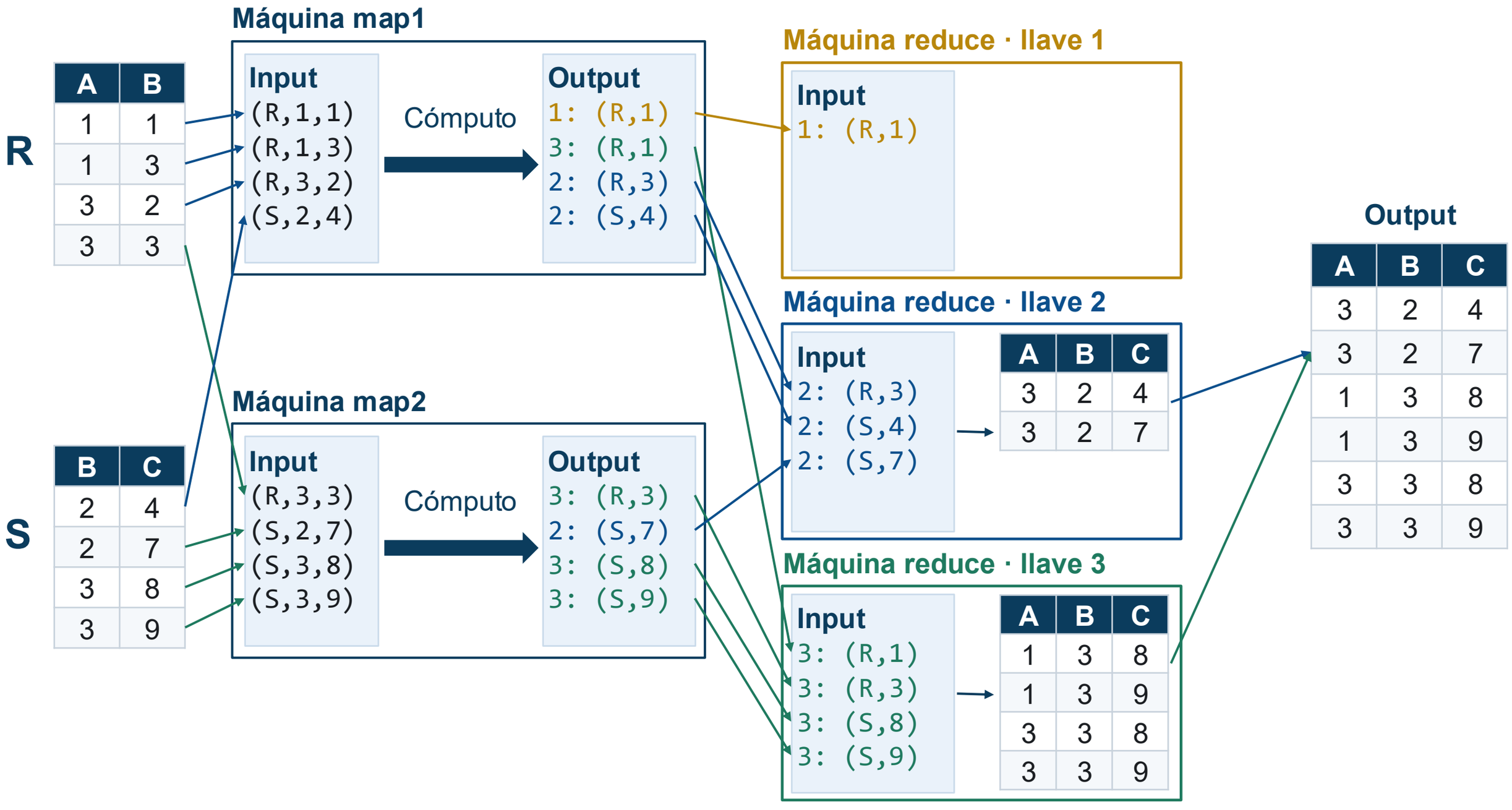
MapReduce – Ejemplo: join

Reduce



MapReduce – Ejemplo: join

Output



MapReduce y BDD

Relevancia

No es un descubrimiento nuevo, pero recientemente se ha observado que se ajusta perfectamente a las necesidades de las grandes BD

Es la arquitectura más importante en sistemas que reciben grandes bases de datos

Apache Hadoop: la implementación open source de MapReduce, presente en muchos sistemas con computación distribuida

Material adicional

Referencias

Video: text search en MongoDB
https://www.youtube.com/watch?v=vR97-4UG7x0

Ramakrishnan y Gehrke, Database Management Systems (3.ª ed.) — Bibliotecas UC
https://bmdigitales-bibliotecas-uc-cl.pucdechile.idm.oclc.org/html5/DATABASE%20MANAGEMENT%20SYSTEMS/961/

